

Digital Verification Course
Project 1

Digital Verification for AMBA AXI4 Interface to a Single Port Synchronous RAM Using SV Class-Based Environment

Submitted to: Eng. Hassan Khaled

Submitted by:
Mahmoud Ismail Mahmoud
Mostafa Mohamed Farhan

Table of Contents

1. System Overview:	6
1.2 Read operation architecture:	6
1.3 Write specifications:	7
1.4 Read specifications:	7
1.5 Boundary checks and address alignment:	7
1.6 Test environment top view:	8
2. Memory Verification:	9
2.1 Testbench:	9
2.1.1 Initial block:	10
2.1.2 Tasks:	11
2.2 Bugs fixing:	13
2.3 The verification flow:	13
2.4 Modularity and scoreboarding:	13
3. Memory Classbased Verification:	14
3.1 Memory Interface:	14
3.2 Memory Transaction	14
3.3 Memory Driver	16
3.4 Memory Generator	17
3.5 Memory Monitor	18
3.6 Memory Scoreboard	19
3.7 Memory Environment	20
3.8 Memory Tb	22
4. AXI Verification Environment:	24
4.1 Configurations package (AXI_pkg.sv):	24
4.1.1 Global parameter configuration:	24
4.1.2 Operation type definition (op_t):	24
4.2 Interface (AXI_intrf.sv):	24
4.2.1 Encapsulation of the 5 AXI channels:	25
4.2.2 Modports for access control & direction safety:	25
4.3 Transaction sequence (AXI_transaction.sv):	25
4.3.1 Constraints:	26
4.3.2 Functional coverage:	27

4.3.3 Signal architecture:	29
4.3.4 Randomization constraints:.....	29
4.3.5 Functional coverage:	29
4.4 Generator components (AXI_generator.sv):.....	29
4.4.1 Structure:.....	30
4.4.1 Transaction instantiation & randomization:.....	30
4.4.2 Functional coverage triggering:	31
4.4.3 Inter-thread communication & handshaking:	31
4.4.4 Completion flag (done):.....	31
4.5 Driver component (AXI_driver.sv):.....	31
4.5.1 Structure:.....	31
4.5.2 Reset initialization sequence	34
4.5.3 Signal timing & alignment strategy	34
4.5.4 Execution states (case (txn.op))	35
4.5.5 Generator handshaking	35
4.6 Monitor component (AXI_monitor.sv):.....	35
4.6.1 Structure:.....	35
4.6.2 Sampling:	36
4.6.3 Handshaking:	36
4.7 Scoreboard component (AXI_scoreboard.sv):.....	37
4.7.1 Golden model logic:.....	37
4.7.2 Main checker:.....	37
4.7.3 Flowchart:	37
4.7.4 Scoreboard conclusion:	39
4.8 Environment wrapper (AXI_env.sv):.....	40
4.8.1 Structure:.....	40
4.9 Top module (AXI_tb.sv):.....	42
4.9.1 Structure:.....	42
4.10 Assertions (AXI_properties.sv):	44
4.10.1 Structure:.....	44
4.10.2 Assertions coverage:	45
5. AXI4 Bugs Fixing:	46
5.1 Problem 1:.....	46

5.2 Problem 2:	46
5.3 Problem 3:	47
5.4 Problem 4:	47
5.5 Problem 5:	47
5.6 Problem 6:	48
6. Testing Results:	49
6.1 Check reset:	49
6.2 Write operation and check priority:	50
6.3 Read operation:	51
6.4 ARVALID = 0 , AWVALID = 0:	52
6.5 Random write operations without 4 KB boundary crossing:	53
6.6 Random read operations without 4 KB boundary crossing:	53
6.7 Random write operations with 4 KB boundary crossing:	54
6.8 Random read operations with 4 KB boundary crossing:	54
6.9 Random operations with and without 4 KB boundary crossing:	55
6.10 Randomization of ARESETn during the simulation:	56
7. Coverage:	57
7.1 Code coverage exclude:	57
7.1.1 FSM exclusion:	57
7.1.2 Toggle exclusion:	57
7.1.3 Line exclusion:	58
7.2 Code coverage:	59
7.3 Functional and cross coverage:	59

Figure 1: Write operation architecture	6
Figure 2: Read operation architecture.....	6
Figure 3: Test environment structure	8
Figure 4: Golden model flowchart.....	37
Figure 5: Writing flowchart	38
Figure 6: Reading flowchart	39
Figure 7: Assertions coverage.....	45
Figure 8: Reset test.....	49
Figure 9: Write operation and priority checking test.	50
Figure 10: Read operation test.	51
Figure 11: IDLE test.	52
Figure 12: Write operation without 4KB boundary crossing test.	53
Figure 13: Random read operations without 4 KB boundary crossing test	53
Figure 14: Random write operations with 4KB boundary crossing test.....	54
Figure 15: Random read operations with 4 KB boundary crossing test.	54
Figure 16: Random operations with and without 4 KB boundary crossing test	55
Figure 17: Randomization of ARESETn during the simulation test.	56

1. System Overview:

AXI4 stands for Advanced eXtensible Interface 4, and it's part of the AMBA (Advanced Microcontroller Bus Architecture) specification developed by ARM. It is a high-frequency system design for communication between Manager and Subordinate components used widely in System-on-Chip (SoC) designs to connect IP blocks.

The AXI protocol is burst-based. Every transaction has address and control information on the address channel that describes the nature of the data to be transferred. The data is transferred between master and slave using a write data channel to the slave or a read data channel to the master. In write transactions, in which all the data flows from the master to the slave, the AXI protocol has an additional write response channel to allow the slave to signal to the master the completion of the write transaction. AXI has two channels, architecture representing write and read operations.

1.1 Write operation architecture:

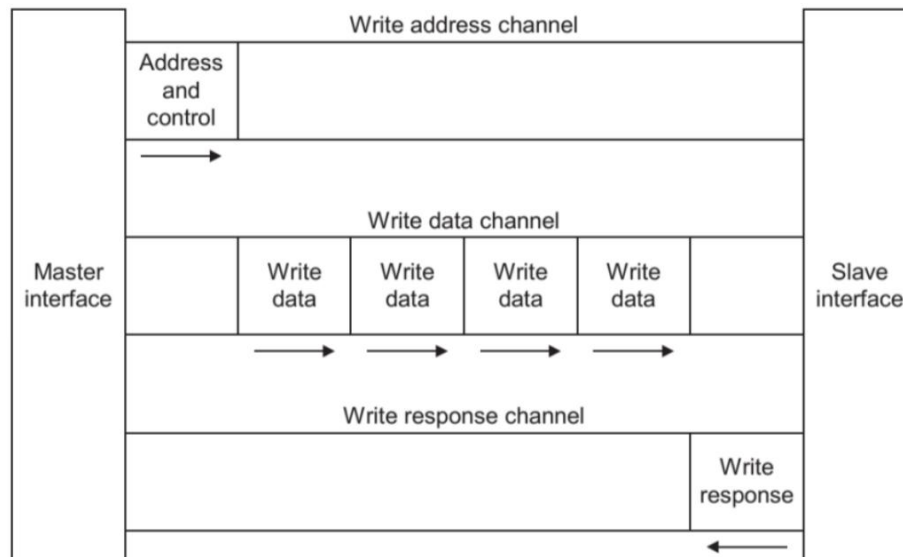


Figure 1: Write operation architecture

1.2 Read operation architecture:

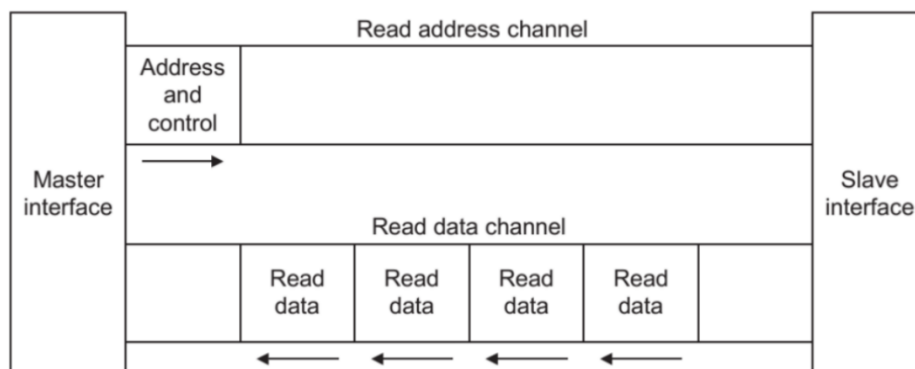


Figure 2: Read operation architecture

1.3 Write specifications:

- *Write address channel:* The master can assert the **AWVALID** signal only when it drives valid address and control information. It must remain asserted until the slave accepts the address and control information and asserts the associated **AWREADY** signal.
- *Write data channel:* During a write burst, the master can assert the **WVALID** signal only when it drives valid write data. **WVALID** must remain asserted until the slave accepts the write data and asserts the **WREADY** signal. The default value of **WREADY** can be HIGH, but only if the slave can always accept writing data in a single cycle. The master must assert the **WLAST** signal when it drives the final write transfer in the burst.
- *Write response channel:* The slave can assert the **BVALID** signal only when it drives a valid write response. **BVALID** must remain asserted until the master accepts the written response and asserts **BREADY**.

1.4 Read specifications:

- *Read address channel:* The master can assert the **ARVALID** signal only when it drives valid address and control information. It must remain asserted until the slave accepts the address and control information and asserts the associated **ARREADY** signal. The default value of **ARREADY** default value is HIGH, although if **ARREADY** is HIGH then the slave must be able to accept any valid address that is presented to it.
- *Read data channel:* The slave can assert the **RVALID** signal only when it drives valid read data. **RVALID** must remain asserted until the master accepts the data and asserts the **RREADY** signal. Even if a slave has only one source of read data, it must assert the **RVALID** signal only in response to a request for the data. The master interface uses the **RREADY** signal to indicate that it accepts the data. The default value of **RREADY** can be HIGH, but only if the master is able to accept reading data immediately, whenever it performs a read transaction. The slave must assert the **RLAST** signal when it drives the final read transfer in the burst.

1.5 Boundary checks and address alignment:

AXI4 supports burst transfers, which can lead to complex address alignment and boundary crossing issues. This design applies compliance checks to ensure proper memory behavior.

1. *4KB Boundary Check:* AXI4 protocol mandates that bursts must not cross 4KB boundaries. Which means the **AWADDR** or **ARADDR** must not exceed the 4KB boundaries to work normally without errors. If it exceeds, the burst is invalid, and a **SLVERR** response is generated.
2. *Memory Range Check:* Internal memory is word addressable. The design shifts address right by 2 ($\gg 2$) to convert from byte address to word address. A range check confirms: $(\text{addr} \gg 2) < \text{MEMORY_DEPTH}$.

1.6 Test environment top view:

A SystemVerilog class-based environment is built for the whole system (AXI4 interface with the single port RAM), and a dedicated testbench for memory unit test. The class-based environment consists of many components wrapped together as the following

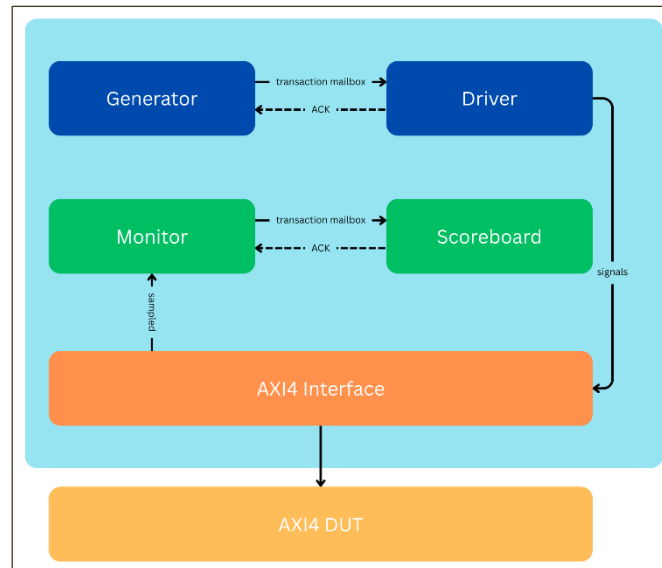


Figure 3: Test environment structure

2. Memory Verification:

Before integrating into the AXI system, initial testing on the standalone memory revealed critical indexing and logic bugs that were resolved prior to full verification.

2.1 Testbench:

```
module axi4_memory_tb;

    parameter DATA_WIDTH = 32;
    parameter ADDR_WIDTH = 10;    // For 1024 locations
    parameter DEPTH = 1024;

    bit clk;
    logic rst_n;
    logic mem_en;
    logic mem_we;
    logic [ADDR_WIDTH-1:0] mem_addr;
    logic [DATA_WIDTH-1:0] mem_wdata;
    logic [DATA_WIDTH-1:0] mem_rdata;

    logic [DATA_WIDTH-1:0] test_wdata_queue[$];
    logic [DATA_WIDTH-1:0] test_rdata_queue[$];
    logic [DATA_WIDTH-1:0] expected_queue[$];

    axi4_memory #(.DATA_WIDTH(DATA_WIDTH), .ADDR_WIDTH(ADDR_WIDTH),
    .DEPTH(DEPTH)) DUT(.*);

    always #5 clk = ~clk;
```

2.1.1 Initial block:

```
initial begin
    clk = 0;
    rst_n = 1;

    $display("\n=====");
    $display("    STARTING AXI4 MEMORY TESTBENCH SIMULATION    ");
    $display("=====");

    // Reset Sequence
    $display("[%0t ns] [RST] Asserting Reset...", $time);
    @(posedge clk);
    rst_n = 0;
    @(posedge clk);
    rst_n = 1;
    expected_queue = '{1024{32'h0}}';
    $display("[%0t ns] [RST] Deasserting Reset. Memory Ready.", $time);

    // -----
    // TEST 1: Write Operations when enable = 0
    // -----
    $display("\n[%0t ns] [WRITE] Starting Write Operations when enable
= 0...", $time);
    for (int i = 0; i < 1024; i++) begin
        write_mem_enableiszero(i, $random, test_wdata_queue);
    end

    $display("\n[%0t ns] [READ] Starting Read Operations...", $time);
    for (int i = 0; i < 1024; i++) begin
        read_mem(i, test_rdata_queue);
    end

    $display("\n[%0t ns] [CHECK] Executing Data Integrity Verification
when enable = 0...", $time);
    checker_data(expected_queue, test_rdata_queue);

    // -----
    // RESET BETWEEN TESTS
    // -----
    $display("\n--- Resetting Queues & Driving Signals for Test 2 ---
");
    test_rdata_queue.delete();
    test_wdata_queue.delete();
    mem_en = 0;
    mem_we = 0;
    @(posedge clk);
```

```

// -----
// TEST 2: Normal Write & Read Operations
// -----
$display("\n[%0t ns] [WRITE] Starting Write Operations (1024
Locations)...", $time);
for (int i = 0; i < 1024; i++) begin
    write_mem(i, $random, test_wdata_queue);
end

$display("\n[%0t ns] [READ] Starting Read Operations (1024
Locations)...", $time);
for (int i = 0; i < 1024; i++) begin
    read_mem(i, test_rdata_queue);
end

$display("\n[%0t ns] [CHECK] Executing Data Integrity
Verification...", $time);
checker_data(test_wdata_queue, test_rdata_queue);

#20;
$display("=====\n");
$stop;

end

```

2.1.2 Tasks:

```

// Use "automatic" and "ref" for dynamic queues to prevent simulator
scoping bugs
task automatic write_mem(
    input logic [ADDR_WIDTH-1:0] mem_addr_t,
    input logic [DATA_WIDTH-1:0] mem_wdata_t,
    ref  logic [DATA_WIDTH-1:0] mem_wdata_queue[$]
);
    @(negedge clk);
    mem_en = 1;
    mem_we = 1;
    mem_addr = mem_addr_t;
    mem_wdata = mem_wdata_t;
    mem_wdata_queue.push_back(mem_wdata_t);
endtask

task automatic write_mem_enableiszero(
    input logic [ADDR_WIDTH-1:0] mem_addr_t,
    input logic [DATA_WIDTH-1:0] mem_wdata_t,
    ref  logic [DATA_WIDTH-1:0] mem_wdata_queue[$]
);

```

```

        @(negedge clk);
        mem_en = 0;
        mem_we = 1;
        mem_addr = mem_addr_t;
        mem_wdata = mem_wdata_t;
        mem_wdata_queue.push_back(mem_wdata_t);
    endtask

    task automatic read_mem(
        input logic [ADDR_WIDTH-1:0] mem_addr_t,
        ref logic [DATA_WIDTH-1:0] mem_rdata_queue[$]
    );
        @(negedge clk);
        mem_en = 1;
        mem_we = 0;
        mem_addr = mem_addr_t;
        @(posedge clk);
        #1;
        mem_rdata_queue.push_back(mem_rdata);
    endtask

    task automatic checker_data(
        ref logic [DATA_WIDTH-1:0] test_wdata_queue_task[$],
        ref logic [DATA_WIDTH-1:0] test_rdata_queue_task[$]
    );
        int pass = 0;
        int fail = 0;

        $display("-----");
        $display(" ADDR      | EXP DATA   | ACT DATA   | STATUS ");
        $display("-----");

        for (int i = 0; i < 1024; i++) begin
            if (test_wdata_queue_task[i] == test_rdata_queue_task[i])
begin
                pass++;
            end else begin
                fail++;
                $display(" 0x%03X | 0x%08X | 0x%08X | [FAIL]", i,
test_wdata_queue_task[i], test_rdata_queue_task[i]);
            end
        end
    end

```

```

$display("-----");
$display("                VERIFICATION RESULTS                ");
$display("-----");
$display("  TOTAL TESTS : %0d", pass + fail);
$display("  PASSED      : %0d", pass);
$display("  FAILED      : %0d", fail);
if (fail == 0) begin
    $display("  OVERALL STATUS: [ SUCCESS ]");
end else begin
    $display("  OVERALL STATUS: [ FAILURE ]");
end
$display("-----");
endtask

endmodule

```

2.2 Bugs fixing:

- *Memory read indexing*: The original implementation had an off-by-one error during data reads.
Original: **mem_rdata** <= memory[**mem_addr-1**].
Corrected: **mem_rdata** <= memory[**mem_addr**].
- *Active-low reset fix*: The reset condition inversion was adjusted to properly handle active-low reset logic.

2.3 The verification flow:

The initial block drives the simulation through a clean, two-phase testing sequence:

- **System Reset**: The simulation begins by asserting and deasserting the active-low **rst_n** signal and zeroing out the **expected_queue**.
- **Test 1 (Write Protection / Enable Test)**: The testbench runs a loop attempting to write random data to all 1024 locations, but it deliberately forces **mem_en** = 0 using the **write_mem_enableiszero** task. It then reads the memory back and compares the results against the **expected_queue** to ensure the memory correctly ignored the write attempts.
- **Inter-Test Reset**: Before moving on, it cleans up by deleting the contents of both the read and write dynamic queues.
- **Test 2 (Normal Operations)**: The testbench enables the memory (**mem_en** = 1) and writes random data across all 1024 addresses using the **write_mem** task. It then reads all locations back and passes the queues to the checker task for verification.

2.4 Modularity and scoreboarding:

- **Stimulus Tasks**: **write_mem**, **write_mem_enable** is zero, and **read_mem** handle the pin-level toggling on the negative edge of the clock. They explicitly pass the queues by reference (ref) to avoid common simulator scoping bugs associated with dynamic arrays.

- The Checker: The **checker_data** task acts as a simple scoreboard. It iterates through the 1024 entries, compares the expected versus actual data, tallies the passes and fails, and neatly prints a formatted verification report to the console.

<pre># ===== # STARTING AXI4 MEMORY TESTBENCH SIMULATION # ===== # [0 ns] [RST] Asserting Reset... # [15 ns] [RST] Deasserting Reset. Memory Ready. # [15 ns] [WRITE] Starting Write Operations when enable = 0... # # [10250 ns] [READ] Starting Read Operations... # # [20496 ns] [CHECK] Executing Data Integrity Verification when enable = 0... # ===== # ADDR EXP DATA ACT DATA STATUS # ===== # # VERIFICATION RESULTS # ===== # TOTAL TESTS : 1024 # PASSED : 1024 # FAILED : 0 # OVERALL STATUS: [SUCCESS] # =====</pre>	<pre>--- Resetting Queues & Driving Signals for Test 2 --- # # [20505 ns] [WRITE] Starting Write Operations (1024 Locations)... # # [30740 ns] [READ] Starting Read Operations (1024 Locations)... # # [40986 ns] [CHECK] Executing Data Integrity Verification... # ===== # ADDR EXP DATA ACT DATA STATUS # ===== # # VERIFICATION RESULTS # ===== # TOTAL TESTS : 1024 # PASSED : 1024 # FAILED : 0 # OVERALL STATUS: [SUCCESS] # =====</pre>
---	---

3. Memory Classbased Verification:

3.1 Memory Interface:

```
4 interface mem_intrf
5     #(parameter DATA_WIDTH = 32,
6       parameter ADDR_WIDTH = 10,    // For 1024 locations
7       parameter DEPTH = 1024)
8     (input logic clk);
9
10    logic rst_n;
11    logic                                mem_en;
12    logic                                mem_we;
13    logic [ADDR_WIDTH-1:0] mem_addr;
14    logic [DATA_WIDTH-1:0] mem_wdata;
15    logic [DATA_WIDTH-1:0] mem_rdata;
16
17    modport TB_SIDE (input clk , mem_rdata,
18                    output rst_n , mem_en , mem_we , mem_addr , mem_wdata);
19
20    modport DUT_SIDE (input clk , rst_n , mem_en , mem_we , mem_addr ,
21                    mem_wdata,
22                    output mem_rdata);
23 endinterface
```

3.2 Memory Transaction

```
package mem_transaction_pkg;
import parameters_pkg::*;

class mem_transaction;
    rand logic rst_n;
    rand logic mem_en;
```

```

rand    logic mem_we;
randc   logic [ADDR_WIDTH-1:0] mem_addr;
rand    logic [DATA_WIDTH-1:0] mem_wdata;
logic   [DATA_WIDTH-1:0] mem_rdata; //sample

// 1. Reset distribution
constraint rst_dist_c {
    rst_n dist {
        0 := 5,
        1 := 95
    };
}

// 2. Memory Enable distribution (Active high: enabled most of the time)
constraint mem_en_dist_c {
    mem_en dist {
        1 := 90,
        0 := 10
    };
}

// 3. Write/Read balance when memory is enabled
constraint mem_we_dist_c {
    mem_we dist {
        1 := 50,
        0 := 50
    };
}

// 4. Memory Address range constraint
constraint mem_addr_range_c {mem_addr inside {[0 : DEPTH - 1]}};

covergroup mem_cg;
    // 1. Reset Transitions & States
    cp_rst_n: coverpoint rst_n;

    // 2. Memory Enable States & Transitions
    cp_mem_en: coverpoint mem_en;

    // 3. Write / Read Operation
    cp_mem_we: coverpoint mem_we {
        bins read_op = {1'b0};
        bins write_op = {1'b1};
        bins wr_to_rd = (1'b1 => 1'b0);
        bins rd_to_wr = (1'b0 => 1'b1);
    }
endgroup

```

```

function new();
    mem_cg = new();
endfunction

function void display_transaction(input string tag = "TXN");

    string op_type;

    if (!rst_n) op_type = "RESET";
    else if (!mem_en) op_type = "IDLE";
    else if (mem_we) op_type = "WRITE";
    else op_type = "READ";

    $display("[%0t][%s] OP=%s | EN=%0b WE=%0b ADDR=0x%03h WDATA=0x%08h |
RDATA=0x%08h", $time,
            tag, op_type, mem_en, mem_we, mem_addr, mem_wdata, mem_rdata);
endfunction
endclass
endpackage

```

3.3 Memory Driver

```

package mem_driver_pkg;

import mem_transaction_pkg::*;

class mem_driver;
    virtual mem_intrf.TB_SIDE vif;
    event drv_done_e;

    // Mailboxes
    mailbox #(mem_transaction) gen2drv_mbx;
    mailbox #(int) drv2gen_mbx;

    task run_driver();
        mem_transaction txn;

        @(negedge vif.clk);
        vif.rst_n = 1'b0;
        vif.mem_en = 1'b0;
        vif.mem_we = 1'b0;
        vif.mem_addr = '0;
        vif.mem_wdata = '0;

        @(negedge vif.clk);
        vif.rst_n <= 1'b1; // Deassert reset
    endtask
endclass

```

```

// 2. Main Driving Loop
forever begin
    gen2drv_mbx.get(txn);

    @(negedge vif.clk);
    vif.rst_n    <= txn.rst_n;
    vif.mem_en   <= txn.mem_en;
    vif.mem_we   <= txn.mem_we;
    vif.mem_addr <= txn.mem_addr;
    vif.mem_wdata <= txn.mem_wdata;

    // Notify monitor and generator
    ->drv_done_e;
    drv2gen_mbx.put(1);
end
endtask
endclass

endpackage

```

3.4 Memory Generator

```

package mem_gen_pkg;

import mem_transaction_pkg::*;

class mem_gen;

    //mailboxes
    mailbox #(mem_transaction) gen2drv_mbx;
    mailbox #(int) drv2gen_mbx;

    int unsigned num_transactions = 20000;
    bit done = 0;

    task run_generator;

        mem_transaction txn;
        int token;

        repeat (num_transactions) begin
            txn = new();

            if (!txn.randomize()) $fatal(1, "[GENERATOR] Randomization failed!");
            txn.mem_cg.sample();
            gen2drv_mbx.put(txn);
            drv2gen_mbx.get(token);
        end
    endtask
endclass

```

```

        end

        done = 1;
        $display("[%0t][GEN] Generator finished. %0d transactions sent.", $time,
num_transactions);
    endtask
endclass
endpackage

```

3.5 Memory Monitor

```

package mem_monitor_pkg;
import mem_transaction_pkg::*;

class mem_monitor;
    virtual mem_intrf.TB_SIDE vif;
    event drv_done_e;
    mailbox #(mem_transaction) mon2scb_mbx;
    mailbox #(int) scb2mon_mbx;

    task run_monitor;

        mem_transaction sampled;
        int token;

        forever
        begin
            @drv_done_e;
            sampled = new();
            sampled.mem_en    = vif.mem_en;
            sampled.mem_we    = vif.mem_we;
            sampled.mem_addr  = vif.mem_addr;
            sampled.mem_wdata = vif.mem_wdata;
            sampled.rst_n     = vif.rst_n;
            @(posedge vif.clk);
            #10ps;
            sampled.mem_rdata = vif.mem_rdata;

            //sampled.display_transaction("MON");

            mon2scb_mbx.put(sampled);
            scb2mon_mbx.get(token);
        end
    endtask
endclass
endpackage

```

3.6 Memory Scoreboard

```

1 package mem_scoreboard_pkg;
2
3 import parameters_pkg::*;
4 import mem_transaction_pkg::*;
5
6 class mem_scoreboard;
7     mailbox #(mem_transaction) mon2scb_mbx;
8     mailbox #(int) scb2mon_mbx;
9
10     logic [DATA_WIDTH-1:0] memory_golden [0:DEPTH-1];
11     logic [DATA_WIDTH-1:0] golden_rdata; // latch rdata form function
12     int pass_count = 0;
13     int fail_count = 0;
14
15     function new();
16         for (int i = 0; i < DEPTH; i++)
17             begin
18                 memory_golden[i] = '0;
19             end
20         golden_rdata = '0;
21     endfunction
22
23     function void golden_mem(
24         input logic rst_n,
25         input logic mem_en,
26         input logic mem_we,
27         input logic [ADDR_WIDTH-1:0] mem_addr,
28         input logic [DATA_WIDTH-1:0] mem_wdata,
29         output logic [DATA_WIDTH-1:0] mem_rdata
30     )
31     if (!rst_n)
32         begin
33             golden_rdata = '0;
34         end else if (mem_en)
35         begin
36             if (mem_we)
37                 memory_golden[mem_addr] = mem_wdata;
38             else
39                 golden_rdata = memory_golden[mem_addr];
40             end
41         mem_rdata = golden_rdata;
42     endfunction
43
44     task run_scoreboard();
45         mem_transaction mon_txn;
46         logic [DATA_WIDTH-1:0] exp_mem_rdata;
47         bit out_ok;
48
49         forever begin
50             mon2scb_mbx.get(mon_txn);
51
52             golden_mem(mon_txn.rst_n, mon_txn.mem_en, mon_txn.mem_we, mon_txn.mem_addr, mon_txn.mem_wdata, exp_mem_rdata);

```

```

54 // 1. Reset Check
55 if (!mon_txn.rst_n)
56     begin
57         out_ok = (mon_txn.mem_rdata === exp_mem_rdata);
58         if (out_ok) begin
59             $display("[%0t][SCB] PASS | OP=RESET ADDR=0x%03h | DUT: 0x%08h | EXP: 0x%08h",
60                 $time, mon_txn.mem_addr, mon_txn.mem_rdata, exp_mem_rdata);
61             pass_count++;
62         end else begin
63             $display("[%0t][SCB] FAIL | OP=RESET ADDR=0x%03h | DUT: 0x%08h | EXP: 0x%08h",
64                 $time, mon_txn.mem_addr, mon_txn.mem_rdata, exp_mem_rdata);
65             fail_count++;
66         end
67     end
68 // 2. Memory Enabled Operations
69 else if (mon_txn.mem_en)
70     begin
71         // Write Transaction
72         if (mon_txn.mem_we)
73             begin
74                 if (memory_golden[mon_txn.mem_addr] === mon_txn.mem_wdata)
75                     begin
76                         $display("[%0t][SCB] PASS | OP=WRITE ADDR=0x%03h | DATA=0x%08h",
77                             $time, mon_txn.mem_addr, mon_txn.mem_wdata);
78                         pass_count++;
79                     end
80                 else
81                     begin
82                         $display("[%0t][SCB] Failed | OP=Write ADDR=0x%03h | DUT: 0x%08h | EXP: 0x%08h",
83                             $time, mon_txn.mem_addr, mon_txn.mem_wdata, memory_golden[mon_txn.mem_addr]);
84                         fail_count++;
85                     end
86             end
87         // Read Transaction Check
88         else
89             begin
90                 out_ok = (mon_txn.mem_rdata === exp_mem_rdata);
91                 if (out_ok)
92                     begin
93                         $display("[%0t][SCB] PASS | OP=READ ADDR=0x%03h | DUT: 0x%08h | EXP: 0x%08h",
94                             $time, mon_txn.mem_addr, mon_txn.mem_rdata, exp_mem_rdata);
95                         pass_count++;
96                     end
97                 else
98                     begin
99                         $display("[%0t][SCB] FAIL | OP=READ ADDR=0x%03h | DUT: 0x%08h | EXP: 0x%08h",
100                             $time, mon_txn.mem_addr, mon_txn.mem_rdata, exp_mem_rdata);
101                         fail_count++;
102                     end
103             end
104     end

```

```

// 3. Memory Disabled / Idle Transaction
else
begin
    $display("[%0t][SCB] PASS | OP=IDLE (mem_en=0)", $time);
    pass_count++;
end

scb2mon_mbx.put(1);
end
endtask

function void report();
    $display("-----");
    $display("          FINAL TEST SUMMARY          ");
    $display("-----");
    $display(" TOTAL PASSED : %0d", pass_count);
    $display(" TOTAL FAILED : %0d", fail_count);
    if (fail_count == 0 && pass_count > 0)
        $display(" STATUS          : TEST PASSED SUCCESSFULLY");
    else
        $display(" STATUS          : TEST FAILED");
    $display("-----");
endfunction

endclass

endpackage

```

3.7 Memory Environment

```

package mem_env_pkg;

import mem_transaction_pkg::*;
import mem_gen_pkg::*;
import mem_driver_pkg::*;
import mem_monitor_pkg::*;
import mem_scoreboard_pkg::*;
import parameters_pkg::*;

class MEM_Env;
    // Components
    mem_gen gen;
    mem_driver drv;
    mem_monitor mon;
    mem_scoreboard scb;

    virtual mem_intrf.TB_SIDE vif;

    // Mailboxes
    mailbox #(mem_transaction) gen2drv_mbx;
    mailbox #(int) drv2gen_mbx;

    mailbox #(mem_transaction) mon2scb_mbx;
    mailbox #(int) scb2mon_mbx;

    task run_env();
        // 1. Create mailboxes (bound size = 1 for back-pressure)

```

```

gen2drv_mbx = new(1);
drv2gen_mbx = new(1);

mon2scb_mbx = new(1);
scb2mon_mbx = new(1);

// 2. Create components
gen = new();
drv = new();
mon = new();
scb = new();
mon.drv_done_e = drv.drv_done_e;
// 3. Connect Mailboxes to components
gen.gen2drv_mbx = gen2drv_mbx;
gen.drv2gen_mbx = drv2gen_mbx;

drv.gen2drv_mbx = gen2drv_mbx;
drv.drv2gen_mbx = drv2gen_mbx;

mon.mon2scb_mbx = mon2scb_mbx;
mon.scb2mon_mbx = scb2mon_mbx;

scb.mon2scb_mbx = mon2scb_mbx;
scb.scb2mon_mbx = scb2mon_mbx;

// 4. Share virtual interface
drv.vif = vif;
mon.vif = vif;

// 5. Fork all tasks concurrently
fork
    gen.run_generator();
    drv.run_driver();
    mon.run_monitor();
    scb.run_scoreboard();

join_any

// Wait extra time after generator finishes, then finish
@(posedge vif.clk);
@(posedge vif.clk);
scb.report();
$stop;
endtask
endclass

endpackage

```

3.8 Memory Tb

```
//
=====
// 8. TOP-LEVEL TESTBENCH MODULE
//
=====
module MEM_tb;

    import parameters_pkg::*;
    import mem_transaction_pkg::*;
    import mem_gen_pkg::*;
    import mem_driver_pkg::*;
    import mem_monitor_pkg::*;
    import mem_scoreboard_pkg::*;
    import mem_env_pkg::*;
    // Clock generation (100 MHz)
    logic clk;
    initial clk = 0;
    always #5 clk = ~clk;

    // Interface instantiation
    mem_intrf intrf (.clk(clk));

    // DUT instantiation
    axi4_memory #(
        .DATA_WIDTH (DATA_WIDTH),
        .ADDR_WIDTH (ADDR_WIDTH),
        .DEPTH      (DEPTH)
    ) DUT (
        .clk      (intrf.clk),
        .rst_n    (intrf.rst_n),
        .mem_en    (intrf.mem_en),
        .mem_we    (intrf.mem_we),
        .mem_addr  (intrf.mem_addr),
        .mem_wdata (intrf.mem_wdata),
        .mem_rdata (intrf.mem_rdata)
    );

    // Test environment
    MEM_Env env;

    initial
    begin
        env = new();
        env.vif = intrf.TB_SIDE;
        env.run_env();
    end
end
```

```
// Waveform dumping
initial begin
    $dumpfile("MEM_tb.vcd");
    $dumpvars(0, MEM_tb);
end
endmodule
```

```
-----
                FINAL TEST SUMMARY
-----
TOTAL PASSED : 20000
TOTAL FAILED : 0
STATUS       : TEST PASSED SUCCESSFULLY
-----
** Note: $stop      : mem_env.sv(71)
    Time: 200025 ns  Iteration: 1  Instance: /MEM_tb
```

4. AXI Verification Environment:

4.1 Configurations package (AXI_pkg.sv):

The *axi_pkg* module serves as the centralized configuration and type-definition package for the entire verification testbench. By defining system-wide parameters and custom enumerated types in a single package, it ensures type safety, parameter consistency, and easy top-level maintainability across all testbench components.

```
package axi_pkg;

    parameter DATA_WIDTH    = 32;
    parameter ADDR_WIDTH    = 16;
    parameter MEMORY_DEPTH = 1024;

    typedef enum {
        WRITE,
        READ,
        IDLE
    } op_t;
endpackage
```

4.1.1 Global parameter configuration:

The package establishes the architectural dimensions of the AXI memory interconnect:

- DATA_WIDTH = 32: Defines the data bus width (32-bit payload per transfer beat).
- ADDR_WIDTH = 16: Configures the address bus width to support up to 64KB of addressable memory space.
- MEMORY_DEPTH = 1024: Sets the depth of the underlying target memory storage array.

4.1.2 Operation type definition (op_t):

To prevent hardcoded magic numbers and improve code readability, the package defines an enumerated type *op_t* representing the three supported transaction operations:

- WRITE: Indicates a write transaction targeting the Write Address (**AW**) and Write Data (**W**) channels.
- READ: Indicates a read transaction targeting the Read Address (**AR**) and Read Data (**R**) channels.
- IDLE: Represents a bus idle cycle where valid signals are deasserted (**AWVALID** = 0, **ARVALID** = 0).

4.2 Interface (AXI_intrf.sv):

The interface module encapsulates all five AXI channels (write address, write data, write response, read address, read data) along with modports for the DUT and Testbench sides.

```

    modport DUT_side (input ACLK, ARESETn, AWADDR, AWLEN, AWSIZE, AWVALID,
WDATA, WVALID, WLAST, BREADY, ARADDR, ARLEN, ARSIZE, ARVALID, RREADY,
    output AWREADY, WREADY, BRESP, BVALID, ARREADY, RDATA, RRESP, RVALID,
RLAST );

    modport TB_side (output ARESETn, AWADDR, AWLEN, AWSIZE, AWVALID, WDATA,
WVALID, WLAST, BREADY, ARADDR, ARLEN, ARSIZE, ARVALID, RREADY, operation,
    input ACLK, AWREADY, WREADY, BRESP, BVALID, ARREADY, RDATA, RRESP,
RVALID, RLAST);
endinterface

```

4.2.1 Encapsulation of the 5 AXI channels:

The interface explicitly declares all signals required by the AXI4 full specification:

- Write Address Channel (AW): Signals driving target write locations, burst length, and valid/ready handshaking (**AWADDR, AWLEN, AWSIZE, AWVALID, AWREADY**).
- Write Data Channel (W): Payload bus carrying actual write values along with last beat indicators (**WDATA, WVALID, WLAST, WREADY**).
- Write Response Channel (B): Feedback channel signaling whether write transactions succeeded or failed (**BRESP, BVALID, BREADY**).
- Read Address Channel (AR): Target read address and burst length control signals (**ARADDR, ARLEN, ARSIZE, ARVALID, ARREADY**).
- Read Data Channel (R): Payload bus receiving data, response status, and burst completion markers (**RDATA, RRESP, RVALID, RLAST, RREADY**).

4.2.2 Modports for access control & direction safety:

To prevent accidental signal assignment collisions during simulation, the interface utilizes two distinct modport configurations:

- DUT Side: Enforces all master/driver outputs as inputs to the design, and slave outputs as outputs from the design.
- TB Side: Configures driver signals as outputs to drive the bus and slave response signals as inputs for sampling.

4.3 Transaction sequence (AXI_transaction.sv):

The class serves as the sequence item (data model) for the object-oriented testbench infrastructure. It encapsulates all data fields across the five AXI channels, specifies procedural constraints for controlled random stimulus generation, and includes an embedded cover group for functional coverage tracking.

4.3.1 Constraints:

```
constraint operation {
    op dist {
        WRITE := 35,
        READ  := 35,
        IDLE  := 30
    };
}

constraint address_mode
{
    addr_mode dist {3:=5, 2:=15, 1:=40, 0:=40};
}

constraint addr {
    if(addr_mode == 0)
    {
        AWADDR inside {[3211 : 4095]};
        ARADDR == AWADDR;
    }
    else if (addr_mode == 2'd1)
    {
        (AWADDR + ((AWLEN + 1) * 4)) <= 4096;
        ARADDR == AWADDR;
    }
    else if (addr_mode == 2'd2)
    {
        AWADDR inside {[4096 : 65536]};
        ARADDR == AWADDR;
    }
    else
    {
        AWADDR inside {65515};
        ARADDR == AWADDR;
    }
}

constraint burst_c {
    AWLEN inside {[0:$]};
    WDATA.size() == AWLEN + 1;
    ARLEN == AWLEN;
}

constraint valid {
    if (op == WRITE) {
        AWVALID == 1;
        ARVALID == AWVALID; // To check Write is High Periority
    } else
```

```

if (op == READ) {
    AWVALID == 0;
    ARVALID != AWVALID;
} else {
    AWVALID == 0;
    ARVALID == 0;
}
}

```

4.3.2 Functional coverage:

```

covergroup cg;

    cp_awaddr: coverpoint AWADDR {
        bins low_addr  = {[0 : 1023]};
        bins mid_addr  = {[1024 : 3071]};
        bins high_addr = {[3072 : 4095]};
    }

    cp_awlen: coverpoint AWLEN {
        bins len_single = {0};           // Single beat
        bins len_short  = {[1 : 15]};   // Short burst
        bins len_med    = {[16 : 127]}; // Medium burst
        bins len_max    = {[128 : 255]}; // Long burst
    }

    cp_araddr: coverpoint ARADDR {
        bins low_addr  = {[0 : 1023]};
        bins mid_addr  = {[1024 : 3071]};
        bins high_addr = {[3072 : 4095]};
    }

    cp_arlen: coverpoint ARLEN {
        bins len_single = {0};
        bins len_short  = {[1 : 15]};
        bins len_med    = {[16 : 127]};
        bins len_max    = {[128 : 255]};
    }

    cp_awvalid: coverpoint AWVALID;
    cp_arvalid: coverpoint ARVALID;
    cp_op: coverpoint op;
    cp_address_mode: coverpoint addr_mode {
        bins mode_0 = {0};
        bins mode_1 = {1};
        bins mode_2 = {2};
    }
}

```

```
cp_op_mode:cross cp_op, cp_address_mode;
cp_wadd_len:cross cp_awaddr, cp_awlen
{
    bins low_singlelen = binsof(cp_awaddr.low_addr) &&
binsof(cp_awlen.len_single);
}
cp_radd_len:cross cp_araddr, cp_arlen
{
    bins low_singlelen = binsof(cp_araddr.low_addr) &&
binsof(cp_arlen.len_single);
}

endgroup
```

4.3.3 Signal architecture:

The class defines variables modeling all standard AXI protocol signals:

- Control & Address Signals: **AWADDR**, **AWLEN**, **AWSIZE**, **AWVALID**, **AWREADY**, **ARADDR**, **ARLEN**, **ARSIZE**, **ARVALID**, and **ARREADY**.
- Data Payloads: Uses dynamic arrays (rand logic `[DATA_WIDTH-1:0] WDATA[]` and `RDATA[]`) to flexibly support single-beat or multi-beat burst transfers.
- Response & Marker Signals: Tracks status responses (**BRESP**, **RRESP**) and burst completion flags (**WLAST**, **RLAST**).

4.3.4 Randomization constraints:

The class enforces strict SystemVerilog constraints to target specific test cases and protocol behaviors:

- Operation Distribution (*operation*): Uses weighted distribution (dist) to generate 35% WRITE, 35% READ, and 30% IDLE transactions.
- Address Range Selection (*address_mode & addr*): Distributes target addresses across 4 modes:
 - Mode 0 (40%): Low/mid address range (`[3211 : 4095]`).
 - Mode 1 (50%): Constrained address range to prevent memory boundary overflow ($((\text{AWADDR} + ((\text{AWLEN} + 1) * 4)) \leq 4096)$).
 - Mode 2 (10%): High memory region (`[4096 : 65536]`).
 - Mode 3 (0%): Fixed high edge address (65515).
- Burst Alignment (*burst_c*): Dynamically resizes the write data array to match the generated burst length (`WDATA.size() == AWLEN + 1`) and locks **ARLEN** to **AWLEN**.
- Channel Valid Assertions (*valid*): Sets **AWVALID** and **ARVALID** according to the active operation type (WRITE, READ, or IDLE).

4.3.5 Functional coverage:

An embedded covergroup tracks verification progress across several cross-coverage metrics:

- Address Bins: Categorizes **AWADDR** and **ARADDR** into `low_addr`, `mid_addr`, and `high_addr` bins.
- Burst Length Bins: Tracks single beats (0), short bursts (1-15), medium bursts (16-127), and maximum bursts (128-255) for both write and read channels.
- Control Bins: Samples coverage for operation types (`cp_op`), address modes (`cp_address_mode`), and valid signals (`cp_awvalid`, `cp_arvalid`).

4.4 Generator components (AXI_generator.sv):

The class forms the stimulus-generation engine of the testbench environment. Its primary objective is to instantiate sequence items (`axi_transaction`), trigger randomization and functional coverage sampling, and securely deliver them to the Driver module using synchronized SystemVerilog mailboxes.

4.4.1 Structure:

```
package axi_generator_pkg;
import axi_transaction_pkg::*;
import axi_pkg::*;

class axi_generator;

// --- Virtual interface -----
virtual AXI_intrf.TB_side vif;

//MailBoxes
mailbox #(axi_transaction) gen2drv_mbx;
mailbox #(int) drv2gen_mbx;

int unsigned num_transaction = 600;
bit done = 0;

task run_gen();
    axi_transaction txn ;
    int token;

    repeat(num_transaction)
    begin
        txn = new();
        if(!txn.randomize())
            $fatal(1,"Generator randomization failed");
        txn.cg.sample();

        // send to driver
        gen2drv_mbx.put(txn);
        drv2gen_mbx.get(token);

    end
    done = 1;
    $display("Gen finished");

endtask //automatic

endclass
endpackage
```

4.4.1 Transaction instantiation & randomization:

- Dynamically constructs a new axi_transaction object inside a loop repeating num_transaction times (configured to 600 iterations).
- Checks the return value of txn.randomize(); if solver conditions fail, it halts simulation immediately via \$fatal to prevent invalid bus operations.

4.4.2 Functional coverage triggering:

- Calls `txn.cg.sample()` on every newly randomized transaction, ensuring every generated stimulus pattern is registered in coverage metrics before execution.

4.4.3 Inter-thread communication & handshaking:

- `gen2drv_mbx` (Generator-to-Driver Mailbox): Transmits the dynamic `axi_transaction` handle to the Driver.
- `drv2gen_mbx` (Driver-to-Generator Mailbox): Uses a blocking `get(token)` call to pause generation until the Driver completes processing the current transaction on the AXI bus. This ping-pong handshaking mechanism guarantees synchronized sequence execution.

4.4.4 Completion flag (done):

- Asserts `done = 1` and prints a completion message upon finishing all 600 transactions.

4.5 Driver component (AXI_driver.sv):

The class acts as the pin-level transactor in the verification testbench. It retrieves high-level `axi_transaction` objects from the generator, converts them into cycle-accurate bus protocols on the virtual interface (vif), and manages protocol handshakes across all five AXI channels.

4.5.1 Structure:

```
package axi_driver_pkg;
import axi_pkg::*;
import axi_transaction_pkg::*;
class axi_driver;
    virtual AXI_intrf.TB_side vif;
    //MailBoxes
    mailbox #(axi_transaction) gen2drv_mbx;
    mailbox #(int) drv2gen_mbx;

    // --- Control signals -----
    shortint i = 0;
    task run_driver();

    axi_transaction txn;

    forever begin
        if(i < 2) begin
            @(negedge vif.ACLK);
            vif.ARESETn    <= 1'b0;

            end
        else
            begin
                if (i == 2)
                    begin
                        vif.ARESETn <= 1'b1;
                        @(negedge vif.ACLK);
                    end

                    gen2drv_mbx.get(txn);
```

```

        vif.ARESETn <= txn.ARESETn;
        @(negedge vif.ACLK);

    if(!vif.ARESETn)
    begin
        // Reset all outputs
        vif.AWREADY <= 1'b1; // Ready to accept address
        vif.WREADY <= 1'b0;
        vif.BVALID <= 1'b0;
        vif.BRESP <= 2'b00;

        vif.ARREADY <= 1'b1; // Ready to accept address
        vif.RVALID <= 1'b0;
        vif.RRESP <= 2'b00;
        vif.RDATA <= {DATA_WIDTH{1'b0}};
        vif.RLAST <= 1'b0;
    end
    else
    begin
        case (txn.op)
            WRITE: begin
                @(negedge vif.ACLK);
                vif.AWVALID <= txn.AWVALID;
                vif.AWADDR <= txn.AWADDR;
                vif.AWLEN <= txn.AWLEN;
                vif.AWSIZE <= 'd2;

                while (!vif.AWREADY)
                    @(posedge vif.ACLK);

                @(negedge vif.ACLK);
                vif.AWVALID <= 0;

                $display("WDATA size = %0d", txn.WDATA.size());

                foreach (txn.WDATA[i]) begin

                    @(negedge
vif.ACLK); // Commented: Delay
insertion.

                    vif.WVALID <= 1;
                    vif.BREADY <= 1;
                    vif.WLAST <= (i == txn.WDATA.size()-1);
                    vif.WDATA <= txn.WDATA[i];

                    while (!vif.WREADY)
                        @(posedge vif.ACLK);

```

```

        @(negedge vif.ACLK);
        vif.WVALID <= 0;
        vif.WLAST  <= 0;

    end

    while (!vif.BVALID)
        @(posedge vif.ACLK);
        txn.BRESP = vif.BRESP;

        @(negedge vif.ACLK);    // keep this – don't remove
it    -- removed for sim cases
        // #50ps;
        vif.BREADY <= 0;

    end

    READ: begin
        @(negedge vif.ACLK);
        //repeat(2) @(posedge vif.ACLK);
        vif.ARADDR <= txn.ARADDR;
        vif.ARLLEN <= txn.ARLLEN;
        vif.ARVALID <= txn.ARVALID;
        vif.ARSIZE <= 'd2;
        vif.AWVALID <= txn.AWVALID;
        vif.AWADDR <= txn.AWADDR;
        vif.AWLEN  <= txn.AWLEN;
        vif.AWSIZE <= 'd2;

        while (!vif.ARREADY)
            @(posedge vif.ACLK);

        @(negedge vif.ACLK);    // making it with
negative edge not the positive edge
        vif.ARVALID <= 0;

        txn.RDATA = new[txn.ARLLEN+1];

        foreach (txn.RDATA[i]) begin

            @(negedge vif.ACLK);
            vif.RREADY <= 1;

            while (!vif.RVALID)
                @(negedge vif.ACLK);

            txn.RDATA[i] = vif.RDATA;
            txn.RRESP    = vif.RRESP;    // capture

```

```

end

    @(negedge vif.ACLK);
    vif.RREADY <= 0;

end

IDLE: begin
    @(negedge vif.ACLK);

    vif.AWVALID <= txn.AWVALID;
    vif.WVALID <= 0;
    vif.BREADY <= 0;
    vif.AWADDR <= txn.AWADDR;
    vif.AWLEN <= txn.AWLEN;
    vif.AWSIZE <= 'd2;

    vif.ARVALID <= txn.ARVALID;
    vif.RREADY <= 0;
    vif.WLAST <= 0;
    vif.ARADDR <= txn.ARADDR;
    vif.ARLLEN <= txn.ARLLEN;
    vif.ARSIZE <= 'd2;

    @(posedge vif.ACLK);

end

endcase
end
    drv2gen_mbx.put(1);
end // new
    i = i + 1; // new
end
endtask
endclass
endpackage

```

4.5.2 Reset initialization sequence

- The driver manages system reset locally during the first two iterations ($i < 2$).
- It asserts active-low reset ($\text{vif.ARESETn} \leq 1'b0$) on negedge vif.ACLK and subsequently deasserts it ($\text{vif.ARESETn} \leq 1'b1$) before pulling transactions from the generator mailbox.

4.5.3 Signal timing & alignment strategy

- Clock Synchronization: Inputs are driven on the falling clock edge (negedge vif.ACLK) to prevent hold-time violations and race conditions at setup/sample edges.
- Handshake Polling: Wait loops (while (! vif.AWREADY), while (! vif.WREADY), while (! vif.BVALID), while (! vif.ARREADY), while (! vif.RVALID)) stall execution

until the DUT acknowledges signals, ensuring full compliance with standard AXI handshake rules.

4.5.4 Execution states (case (txn.op))

- WRITE Transactions:
 1. Drives write address control signals (**AWADDR**, **AWLEN**, **AWVALID**, **AWSIZE**) and waits for **AWREADY**.
 2. Iterates over the **WDATA** array using a foreach loop. Drives individual data beats (**WDATA[i]**), asserts **WVALID**, and conditionally asserts **WLAST** on the final beat ($i == \text{txn.WDATA.size}() - 1$).
 3. Asserts **BREADY** and waits for **BVALID** from the slave to sample write response status (**BRESP**).
- READ Transactions:
 1. Drives read address parameters (**ARADDR**, **ARLEN**, **ARVALID**, **ARSIZE**) and waits for **ARREADY**.
 2. Dynamically allocates the **RDATA** array based on **ARLEN** + 1.
 3. Asserts **RREADY** and loops through expected read beats, storing incoming **RDATA** and **RRESP**.
 4. Validates **RLAST** assertion: confirms **RLAST** is present on the final beat and flags errors if missing or asserted prematurely.
- IDLE Transactions:
 1. Sets valid flags (**WVALID**, **BREADY**, **RREADY**) to 0 to keep the bus inactive during idle cycles.

4.5.5 Generator handshaking

- After completing transaction execution, the driver sends an integer token (`drv2gen_mbx.put(1)`) back to the generator, unblocking the generator thread for the next sequence item.

4.6 Monitor component (AXI_monitor.sv):

The monitoring is built on sampling every cycle, input and outputs for the DUT. And then passes the transaction to the score board.

4.6.1 Structure:

```
task run_monitor();

    axi_transaction sampled;
    int token;

    forever begin
        @(posedge vif.ACLK);
        #100ps;

        sampled = new();

        sampled.ARESETn = vif.ARESETn;
```

```

sampled.AWADDR = vif.AWADDR;
sampled.AWLEN  = vif.AWLEN;
sampled.AWSIZE = vif.AWSIZE;
sampled.AWVALID = vif.AWVALID;

sampled.WDATA  = new[1];
sampled.WDATA_MON = vif.WDATA;
sampled.WVALID = vif.WVALID;
sampled.WLAST  = vif.WLAST;

sampled.BREADY = vif.BREADY;

sampled.ARADDR = vif.ARADDR;
sampled.ARLen  = vif.ARLen;
sampled.ARSIZE = vif.ARSIZE;
sampled.ARVALID = vif.ARVALID;

sampled.RREADY = vif.RREADY;

sampled.AWREADY = vif.AWREADY;
sampled.WREADY  = vif.WREADY;
sampled.BRESP   = vif.BRESP;
sampled.BVALID  = vif.BVALID;
sampled.ARREADY = vif.ARREADY;

sampled.RDATA  = new[1];
sampled.RDATA_MON = vif.RDATA;
sampled.RRESP  = vif.RRESP;
sampled.RVALID = vif.RVALID;
sampled.RLAST  = vif.RLAST;

mon2scb_mbx.put(sampled);
scb2mon_mbx.get(token);

end
endtask
endclass
endpackage

```

4.6.2 Sampling:

- Sampling is done at every positive + 100ps to make sure that values are settled.

4.6.3 Handshaking:

- Handshaking is done with the scoreboard, to make sure that sampled transaction arrives to the scoreboard every clk cycle.

4.7 Scoreboard component (AXI_scoreboard.sv):

The scoreboard functionalities including the golden model task, checker task, and summary report task.

4.7.1 Golden model logic:

The logic is built according to the AXI4 specifications as following:

1. Main if-statement for **ARESETn**, its else include the main functionality.
2. Main case-statement (case(CS_W)) for writing operation including internally a case-statement (case(CS_R)) for reading operations.
3. The internal logic including boundary checking for addressing, **AREADY**, **AWREADY** assertion, **RLAST**, **WLAST** assertion, and response channels assertions.

4.7.2 Main checker:

Ask if the actual values match expected values from the golden model every clock cycles.

4.7.3 Flowchart:

This flowchart describes the golden model behavior.

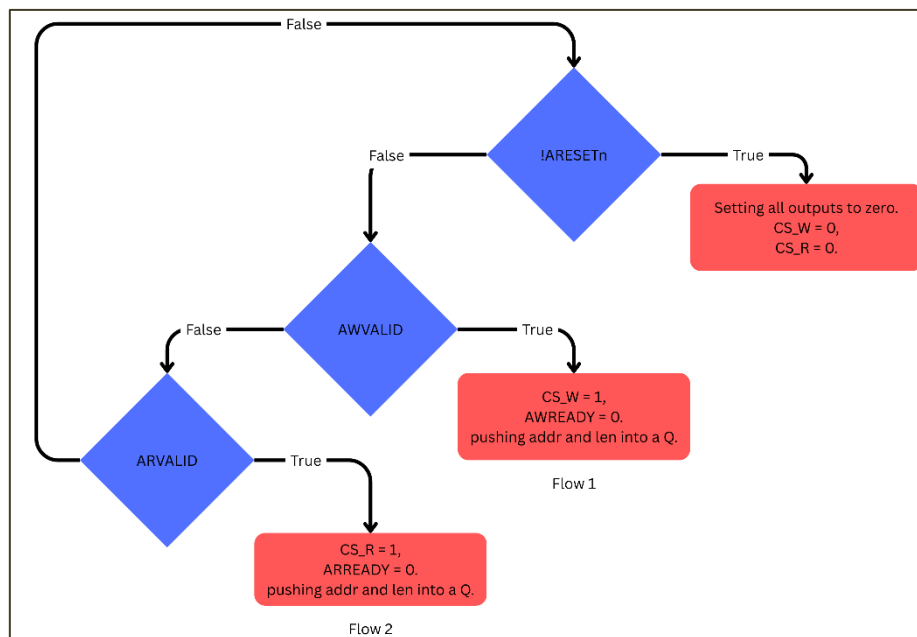


Figure 4: Golden model flowchart

Each flow describes the operations done for writing and reading. Making sure that the golden model triggered each clock cycle, so the it always do the same process and asks for **ARESETn** each clock cycle.

Flow 1 flowchart:

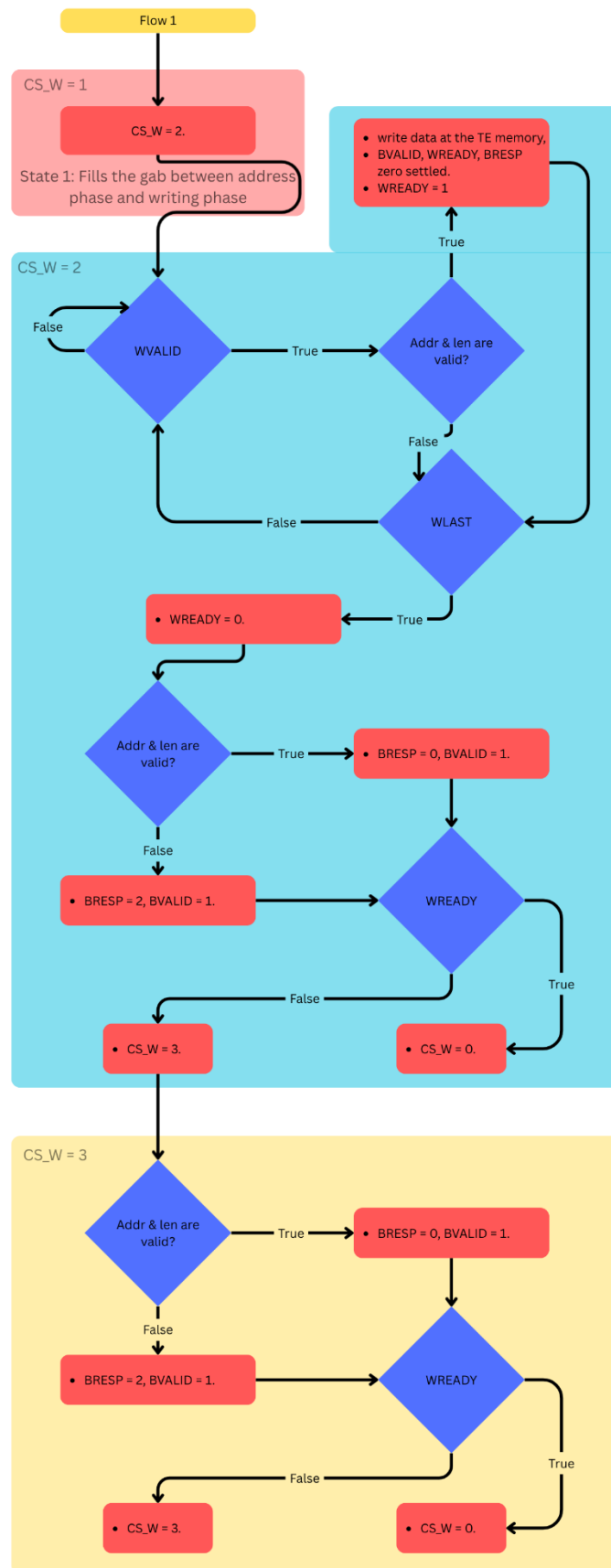


Figure 5: Writing flowchart

Flow 2 flowchart:

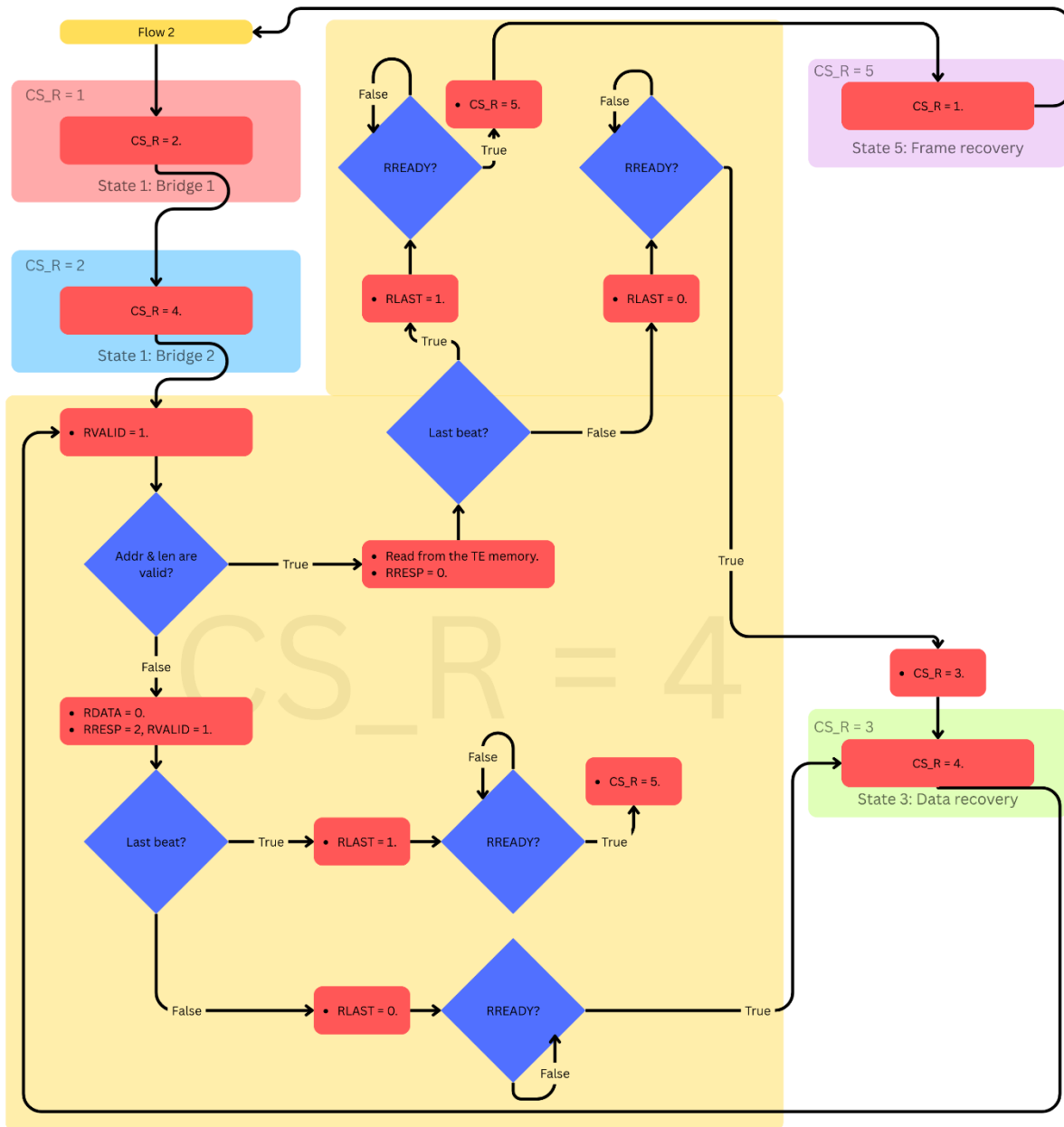


Figure 6: Reading flowchart

4.7.4 Scoreboard conclusion:

The scoreboard was built by logic of “Testing all input signals at the positive and compare between the expected and actual values at the same time”.

The pass rate metric is the number of passed clock cycles.

4.8 Environment wrapper (AXI_env.sv):

This class wrapped the whole components into one package, and runs all tasks in threads behavior, finally it prints the summary report.

4.8.1 Structure:

```
package AXI_env_pkg;

import axi_pkg::*;
import axi_transaction_pkg::*;
import axi_generator_pkg::*;
import axi_driver_pkg::*;
import AXI_monitor_pkg::*;
import AXI_scoreboard_pkg::*;

class AXI_env;

    // --- Components -----
    axi_generator    gen;
    axi_driver       drv;
    AXI_monitor      mon;
    AXI_scoreboard   scb;

    // --- Virtual interface -----
    virtual AXI_intrf.TB_side    vif;

    // --- Mail boxes -----
    // -- GEN to DRV -----
    mailbox #(axi_transaction)  gen2drv_mbx;
    mailbox #(int)               drv2gen_mbx;

    // -- MON to SCB -----
    mailbox #(axi_transaction)  mon2scb_mbx;
    mailbox #(int)               scb2mon_mbx;

    task run_env();

        // --- Mail boxes creation -----
        gen2drv_mbx = new(1);
        drv2gen_mbx = new(1);
        mon2scb_mbx = new(1);
        scb2mon_mbx = new(1);

        // --- Components creation -----
        gen      = new();
        drv      = new();
        mon      = new();
        scb      = new();
    endtask
endclass
```

```

// --- Wiring generator mail boxes -----
gen.gen2drv_mbx      = gen2drv_mbx;
gen.drv2gen_mbx      = drv2gen_mbx;

// --- Wiring driver mail boxes -----
drv.gen2drv_mbx      = gen2drv_mbx;
drv.drv2gen_mbx      = drv2gen_mbx;

// --- Wiring monitor mail boxes -----
mon.mon2scb_mbx      = mon2scb_mbx;
mon.scb2mon_mbx      = scb2mon_mbx;

// --- Wiring scoreboard mail boxes -----
scb.mon2scb_mbx      = mon2scb_mbx;
scb.scb2mon_mbx      = scb2mon_mbx;

// --- Sharing the virtual interface -----
drv.vif              = vif;
mon.vif              = vif;

// --- Fork all tasks -----
fork
    gen.run_gen();
    drv.run_driver();
    mon.run_monitor();
    scb.run_scoreboard();
join_any

// --- Waiting 2 cycles before finish -----
@(posedge vif.ACLK);
@(posedge vif.ACLK);

scb.sum_report();
$stop;
endtask
endclass
endpackage

```

4.9 Top module (AXI_tb.sv):

4.9.1 Structure:

```
`timescale 1ns/1ps

import axi_pkg::*;
import axi_transaction_pkg::*;
import axi_generator_pkg::*;
import axi_driver_pkg::*;
import AXI_monitor_pkg::*;
import AXI_scoreboard_pkg::*;
import AXI_env_pkg::*;

module AXI_tb;

    // --- clock generation -----
    logic ACLK;
    initial
        ACLK = 1;
    always #5 ACLK = ~ACLK;

    // --- interface instantiation -----
    AXI_intrf #(
        .DATA_WIDTH(DATA_WIDTH),
        .ADDR_WIDTH(ADDR_WIDTH),
        .MEMORY_DEPTH(MEMORY_DEPTH)
    ) intrf (
        .ACLK(ACLK)
    );

    // --- DUT instantiation -----
    axi4 DUT (
        .ACLK      (ACLK),
        .ARESETn   (intrf.ARESETn),

        // Write Address
        .AWADDR    (intrf.AWADDR),
        .AWLEN     (intrf.AWLEN),
        .AWSIZE    (intrf.AWSIZE),
        .AWVALID   (intrf.AWVALID),
        .AWREADY   (intrf.AWREADY),

        // Write Data
        .WDATA     (intrf.WDATA),
        .WVALID    (intrf.WVALID),
        .WLAST     (intrf.WLAST),
        .WREADY    (intrf.WREADY),
```

```

        // Write Response
        .BRESP      (intrf.BRESP),
        .BVALID     (intrf.BVALID),
        .BREADY     (intrf.BREADY),

        // Read Address
        .ARADDR     (intrf.ARADDR),
        .ARLEN      (intrf.ARLEN),
        .ARSIZE     (intrf.ARSIZE),
        .ARVALID    (intrf.ARVALID),
        .ARREADY    (intrf.ARREADY),

        // Read Data
        .RDATA      (intrf.RDATA),
        .RRESP      (intrf.RRESP),
        .RVALID     (intrf.RVALID),
        .RLAST      (intrf.RLAST),
        .RREADY     (intrf.RREADY)
    );

    // --- Environment -----
    AXI_env env;

    initial begin
        env          = new();
        env.vif      = intrf.TB_side;

        intrf.ARESETn = 0;
        @(posedge ACLK);
        @(posedge ACLK);
        @(posedge ACLK);
        intrf.ARESETn = 1;
        @(posedge ACLK);

        env.run_env();
    end

    initial begin
        $dumpfile("AXI_tb.vcd");
        $dumpvars(0, AXI_tb);
    end
endmodule

```

4.10 Assertions (AXI_properties.sv):

Assertion was combined with the scoreboard to work in parallel, to maintain the best checking results.

4.10.1 Structure:

```
package AXI_properties_pkg;

    property p_bvalid_after_wlast(ACLK,ARESETn,BVALID,WVALID,WREADY,WLAST);
    @(posedge ACLK) disable iff (!ARESETn)
    $rose(BVALID) |-> $past(WVALID && WREADY && WLAST);
    endproperty

    property RREADY_after_wlast_is_zero(ACLK,ARESETn,RLAST,RREADY,RVALID);
    @(posedge ACLK) disable iff (!ARESETn)
    (RLAST && RREADY && RVALID) |-> ##1 (!RREADY);
    endproperty

    property awready_after_awvalid(ACLK,ARESETn,AWVALID,AWREADY);
    @(posedge ACLK) disable iff (!ARESETn)
    AWVALID |-> ##1 (!AWREADY);
    endproperty

    property WLAST_AND_RLAST(ACLK,ARESETn,WLAST,RLAST);
    @(posedge ACLK) disable iff (!ARESETn)
    not (WLAST && RLAST);
    endproperty

    property WLAST_AND_AWREADY(ACLK,ARESETn,WLAST,AWREADY);
    @(posedge ACLK) disable iff (!ARESETn)
    not (WLAST && AWREADY);
    endproperty

    property RLAST_AND_ARREADY(ACLK,ARESETn,RLAST,ARREADY);
    @(posedge ACLK) disable iff (!ARESETn)
    not (RLAST && ARREADY);
    endproperty

    property WLAST_AND_AWVALID(ACLK,ARESETn,WLAST,AWVALID);
    @(posedge ACLK) disable iff (!ARESETn)
    not (WLAST && AWVALID);
    endproperty

    property RLAST_AND_ARVALID(ACLK,ARESETn,RLAST,ARVALID);
    @(posedge ACLK) disable iff (!ARESETn)
    not (RLAST && ARVALID);
    endproperty

endpackage
```

- **BVALID** after **BLAST** property: Ensures that the **BVALID** signal is asserted after the assertion of **WLAST**, **WREADY**, and **WVALID**.
- **RREADY** after **RLAST** property: Ensures that the **RREADY** will be zero after the end of the frame.
- **AWREADY** after **AWVALID** property: Ensures that the **AWREADY** will be zero after one clock cycle of **AWVALID** de-assertion.
- **WLAST** and **RLAST** property: Ensures that the **RLAST** will not come with the **WLAST** at the same positive edge.
- **WLAST** and **AWREADY** property: Ensures that the **WLAST** will not come with the **AWREADY** at the same positive edge.
- **RLAST** and **ARREADY** property: Ensures that the **RLAST** will not come with the **ARREADY** at the same positive edge.
- **WLAST** and **AWVALID** property: Ensures that the **WLAST** will not come with the **AWVALID** at the same positive edge.
- **RLAST** and **ARVALID** property: Ensures that the **RLAST** will not come with the **ARVALID** at the same positive edge.

After that we assert the properties inside the interface structure.

4.10.2 Assertions coverage:

1					
2	ASSERTION RESULTS:				
3	-----				
4	Name	File(Line)			Failure Pass
5					Count Count
6	-----				
7	/AXI_tb/intrf/Assertion_bvalid_after_last				
8		AXI_intrf.sv(58)		0	272
9	/AXI_tb/intrf/Assertion_rready_after_last				
10		AXI_intrf.sv(62)		0	243
11	/AXI_tb/intrf/Assertion_awready_after_awvalid				
12		AXI_intrf.sv(66)		0	272
13	/AXI_tb/intrf/Assertion_WLAST_AND_RLAST				
14		AXI_intrf.sv(70)		0	29878
15	/AXI_tb/intrf/Assertion_WLAST_AND_AWREADY				
16		AXI_intrf.sv(74)		0	29878
17	/AXI_tb/intrf/Assertion_RLAST_AND_ARREADY				
18		AXI_intrf.sv(78)		0	29878
19	/AXI_tb/intrf/Assertion_WLAST_AND_AWVALID				
20		AXI_intrf.sv(82)		0	29878
21	/AXI_tb/intrf/Assertion_RLAST_AND_ARVALID				
22		AXI_intrf.sv(86)		0	29878

Figure 7: Assertions coverage.

5. AXI4 Bugs Fixing:

5.1 Problem 1:

Since the address increment calculation is based on shift operations derived from `write_size` and `read_size` (where **AXSIZE** represents bytes per transfer beat, capped at $2^2 = 4$ bytes for a 32-bit data width), the effective value of the increment only ranges from 0 to 4 (binary 000 to 100).

Problem: Declaring a full 16-bit wire meant that the upper 13 bits ([15:3]) remained permanently tied to 0. During code coverage and toggle coverage analysis, these upper unused bits created un-reachable "dead bits" that never toggled, preventing the simulation environment from achieving 100% toggle coverage.

```
wire [2:0] write_addr_incr, read_addr_incr; // Edit from wire [ADDR_WIDTH-1:0] write_addr_incr, read_addr_incr; to [2:0]
```

5.2 Problem 2:

According to the AXI4 specification, a single transaction burst must never cross a 4KB address boundary (12'hFFF).

Problem : Originally, boundary checks evaluated the dynamically incrementing address (`write_addr` or `read_addr`) beat-by-beat during the data transfer phase (`W_DATA` / `R_DATA`). This approach had significant drawbacks:

- Late Error Detection: A burst would execute partially before detecting a boundary crossing on later beats.

Applied Fix: To optimize boundary checking, dedicated registers (`write_addr_F` and `read_addr_F`) were introduced to snapshot the initial base address during the address handshake phase (`W_IDLE` / `R_IDLE`). Instead of checking the address dynamically on every clock cycle, the boundary check logic now calculates the total transaction span upfront using the burst length (`write_burst_len` / `read_burst_len`).

```
// Address boundary check (4KB boundary = 12 bits)
assign write_boundary_cross = ((write_addr_F & 12'hFFF) + ((write_burst_len + 1) << write_size)) > 12'hFFF; // EDITED |
assign read_boundary_cross = ((read_addr_F & 12'hFFF) + ((read_burst_len + 1) << read_size)) > 12'hFFF; // EDITED
```

```
// -----
// Write Channel FSM
// -----
case (write_state)
  W_IDLE: begin
    AWREADY <= 1'b1;
    WREADY <= 1'b0;
    BVALID <= 1'b0;

    if (ARVALID && ARREADY) begin
      // Capture address phase information
      write_addr <= AWADDR;
      write_addr_F <= AWADDR; // Added
      write_burst_len <= AWLEN;
      write_burst_cnt <= AWLEN;
      write_size <= AWSIZE;

      AWREADY <= 1'b0;
      write_state <= W_ADDR;
    end
  end
```

```
case (read_state)
  R_IDLE: begin
    ARREADY <= 1'b1;
    RVALID <= 1'b0;
    RLAST <= 1'b0;

    if (ARVALID && ARREADY) begin
      // Capture address phase information
      read_addr <= ARADDR;
      read_addr_F <= ARADDR; // Added
      read_burst_len <= ARLEN;
      read_burst_cnt <= ARLEN;
      read_size <= ARSIZE;

      ARREADY <= 1'b0;
      read_state <= R_ADDR;
    end
  end
```

5.3 Problem 3:

The state variables (write_state and read_state) were originally declared as 3-bit registers (reg [2:0]) while only managing 4 distinct states (00, 01, 10, 11). This left the highest bit permanently unused, creating un-reachable states in the synthesis state table and dropping code coverage.

Applied Fix : Resized state registers from 3 bits to 2 bits (reg [1:0]).

```
// FSM states
reg [1:0] write_state; //Edit to two bits
localparam W_IDLE = 3'd0, // 00
            W_ADDR = 3'd1, // 01
            W_DATA = 3'd2, // 10
            W_RESP = 3'd3; // 11

reg [1:0] read_state; // Edit to two bits
localparam R_IDLE = 3'd0,
            R_ADDR = 3'd1,
            R_WAIT = 3'd2, // Added bec mem is synch
            R_DATA = 3'd3;
```

5.4 Problem 4:

The memory module (axi4_memory) uses a synchronous read interface, requiring one full clock cycle after asserting mem_en and mem_addr before valid data appears on mem_rdata. The original FSM transitioned directly from address sampling (R_ADDR) to data presentation (R_DATA), causing the read channel to output stale or invalid data on the first clock cycle.

Applied Fix : Introduced an intermediate state R_WAIT (2'd2) into the read FSM sequence: R_IDLE → R_ADDR (Assert Address) → R_WAIT (Clock Latency) → R_DATA (Drive Bus)

```
R_ADDR: begin
    // Start first read
    if (read_addr_valid && !read_boundary_cross) begin
        mem_en <= 1'b1;
        mem_addr <= read_addr >> 2; // Convert to word address
    end
    read_state <= R_WAIT;
end
R_WAIT: begin
    read_state <= R_DATA;
end
```

5.5 Problem 5:

A 32-bit register mem_rdata_reg was present in the declarations and reset block but was never assigned or used in the active data path.

Applied Fix : Removed mem_rdata_reg completely:

100	// Registered memory read data for timing
101	//reg [DATA_WIDTH-1:0] mem_rdata_reg; >> // we must remove we don't use
102	
136	
137	// mem_rdata_reg <= {DATA_WIDTH{1'b0}}; // Removed
138	

5.6 Problem 6:

In multi-beat burst transfers over a synchronous memory interface, the memory requires one clock cycle latency after asserting `mem_en` and `mem_addr` before valid read data appears on `mem_rdata`.

Originally, memory read controls (`mem_en` and `mem_addr`) were only triggered after a successful handshake occurred (if (**RREADY** && **RVALID**)) inside the burst loop. This created a timing misalignment:

- When **RVALID**, **RREADY**, and **RLAST** were asserted, **RDATA** would fail to update synchronously because the memory fetch for the next beat was delayed by a clock cycle.
- This resulted in outputting stale data or missing synchronization on burst beats.

Applied Fix : To resolve this latency gap, memory prefetching logic was added at the very beginning of the `R_DATA` state execution (outside the handshake check):

```
R_DATA: begin
    // Present read data
    if (read_addr_valid && !read_boundary_cross) begin
        RDATA <= mem_rdata;
        mem_en <= 1'b1;           // Added.
        mem_addr <= (read_addr + read_addr_incr) >> 2; // Added.
        RRESP <= 2'b00; // OKAY
    end else begin
        RDATA <= {DATA_WIDTH{1'b0}};
        RRESP <= 2'b10; // SLVERR
    end
end
```

6. Testing Results:

6.1 Check reset:

The scoreboard performs a check at every clock cycle. Therefore, even for a small number of transactions, many checks are performed, resulting in a high total check count. This provides continuous cycle-by-cycle verification of DUT behavior.

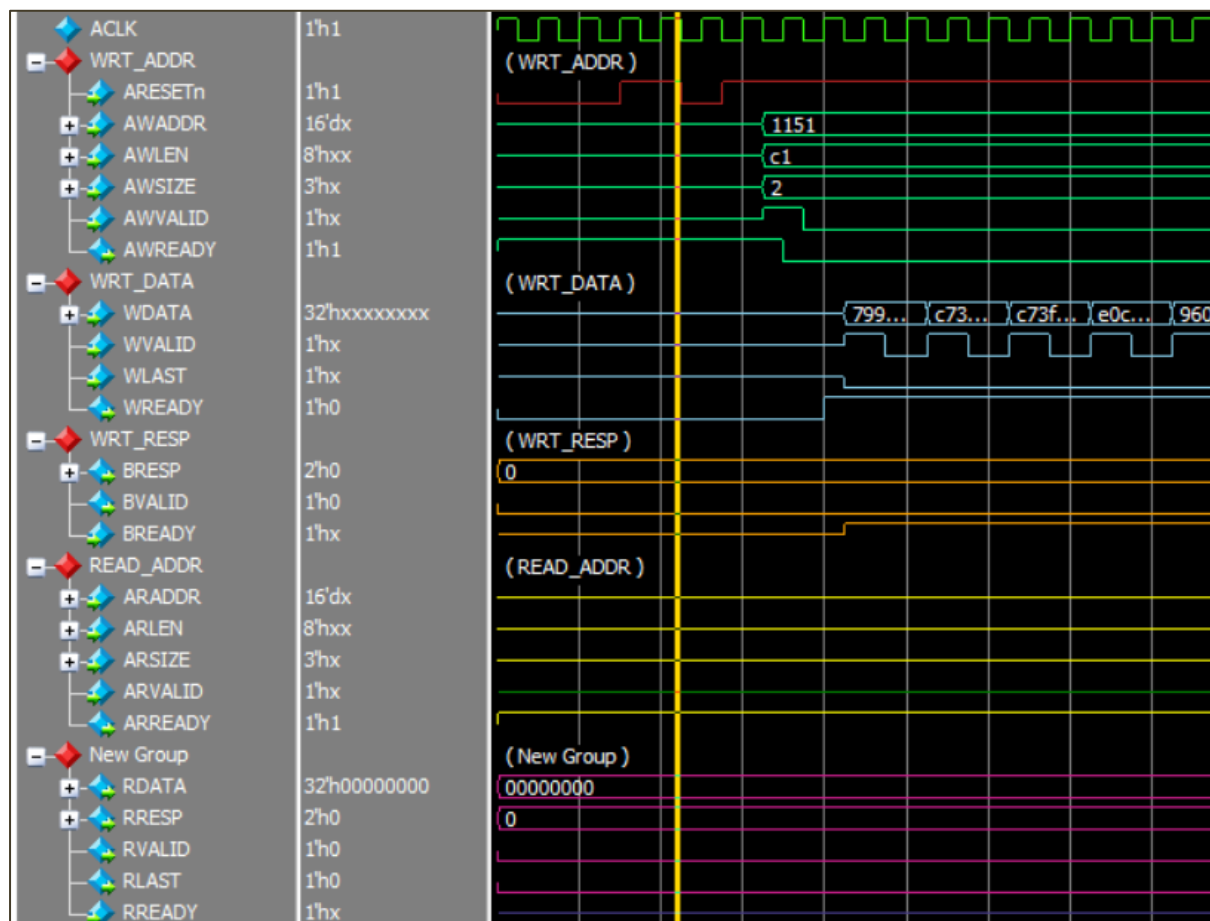


Figure 8: Reset test.

- The waveform shows that ARESETn is initially asserted low, resetting the AXI interface. During reset, the AXI control and response signals remain in their reset/idle states. After ARESETn is deasserted high, the interface exits reset and begins normal operation, with the write state transitioning from the reset state to the appropriate operating states. The reset behavior is correct and matches the expected design behavior. Test case passed as expected.

6.2 Write operation and check priority:

For **ARVALID** = 1, and **AWVALID** = 1.

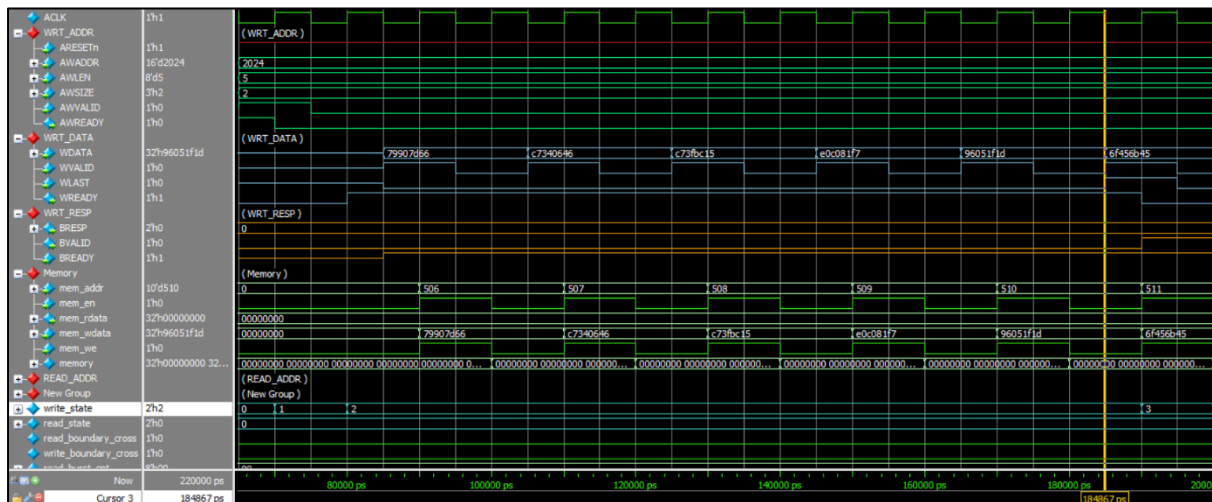


Figure 9: Write operation and priority checking test.

The waveform shows a successful AXI write burst transaction:

- **AWADDR** is set to 2024, indicating the starting write address.
- **AWLEN** is 5, indicating a burst of 6 data beats (**AWLEN** + 1).
- **AWSIZE** is 2, indicating 4 bytes per beat.
- **AWVALID** is asserted while **AWREADY** is high, completing the write-address handshake.
- **WDATA** transfers six data values sequentially: 79907D66, C7340646, C73BFC15, E0C081F7, 96051F1D, and 64566445.
- **WVALID** is asserted for the valid data transfers, while **WREADY** remains high, allowing each data beat to be accepted.
- **WLAST** is asserted on the final data beat, correctly indicating the end of the burst.
- The memory address increments sequentially from 506 to 511.
- **mem_wdata** receives the corresponding write data for each address.
- **mem_we** is asserted during the write operations, enabling the memory to store the transferred data.
- **BRESP** remains 0, indicating an OKAY response.
- **BVALID** is asserted after completion of the write burst, and **BREADY** is high, completing the write-response handshake.
- The write state progresses through the expected states and returns to the idle state after the transaction.

Result: The write burst is completed correctly, with all six data beats written to consecutive memory locations and the expected AXI response generated. Test case passed as expected.

6.3 Read operation:

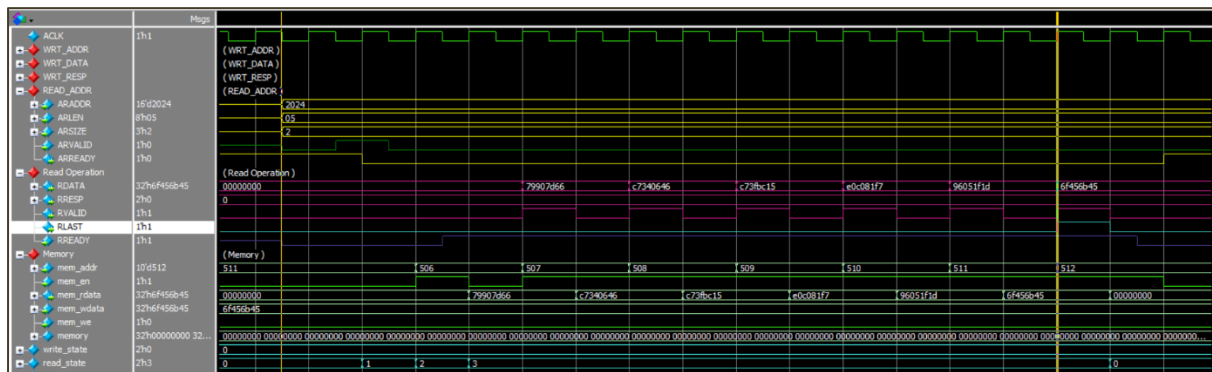


Figure 10: Read operation test.

The waveform shows a successful AXI read burst transaction:

- **ARADDR** is set to 2024, indicating the starting read address.
- **ARLEN** is 05, indicating a burst of 6 read data beats (**ARLEN** + 1).
- **ARSIZE** is 2, indicating 4 bytes per beat.
- **ARVALID** is asserted while **ARREADY** is high, completing the read-address handshake.
- **RDATA** returns six data values sequentially: 79907D66, C7340646, C73BFC15, E0C081F7, 96051F1D, and 645664B5.
- **RVALID** is asserted for each valid read-data transfer.
- **RREADY** is high, allowing the master to accept each returned data beat.
- **RLAST** is asserted on the final data beat, correctly indicating the end of the read burst.
- The memory addresses progress sequentially from 506 to 511.
- **mem_rdata** provides the corresponding stored data from each memory location.
- **RRESP** remains 0, indicating an OKAY response.
- The read state progresses through the expected states and returns to the idle state after the complete read transaction.

Result: The read burst is completed correctly, with all six data beats retrieved from consecutive memory locations and transferred to the master in the expected order. Test case passed as expected.

6.4 ARVALID = 0 , AWVALID = 0:

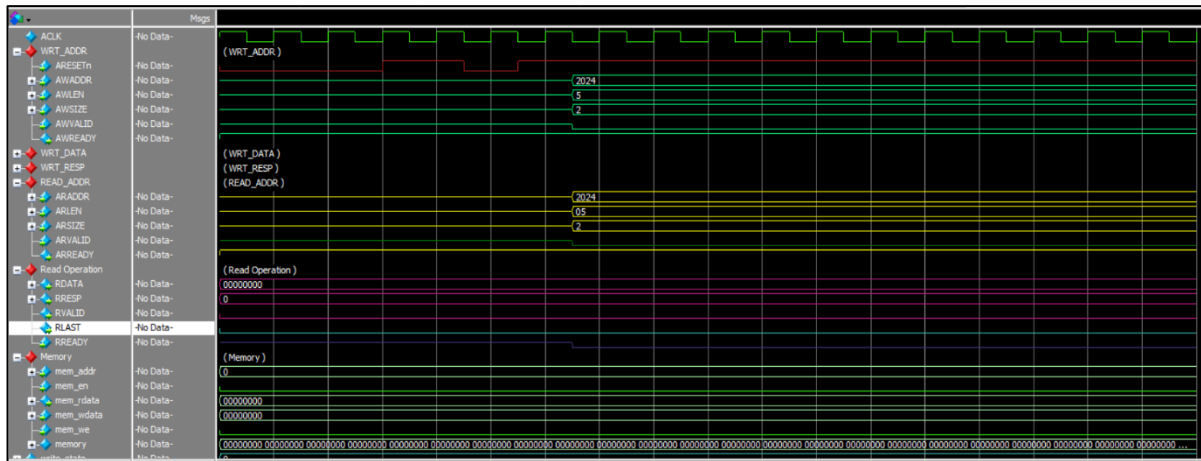


Figure 11: IDLE test.

The waveform verifies that no AXI read or write transaction is initiated when both valid signals are deasserted:

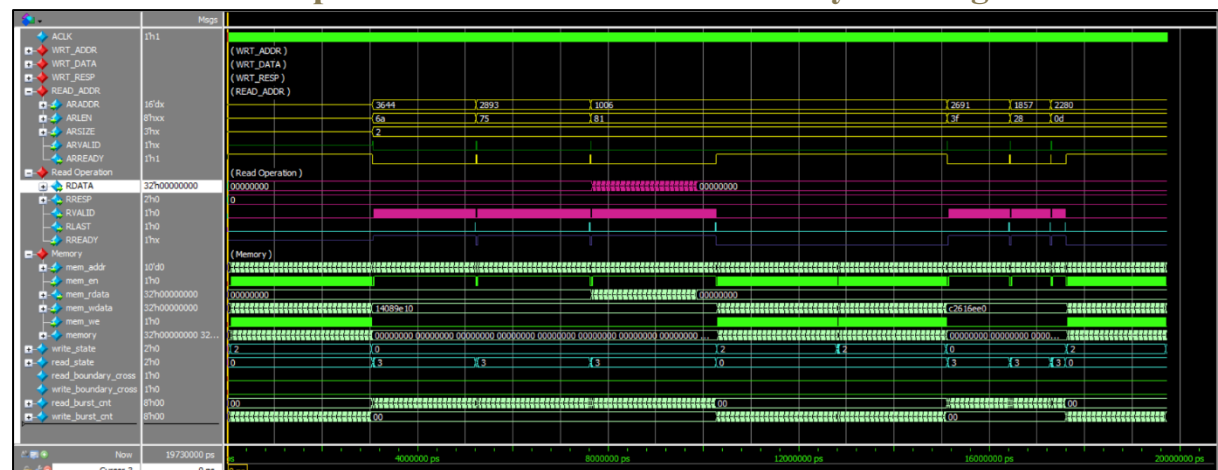
- **AWVALID** = 0, so the write address channel does not initiate a write transaction.
- **ARVALID** = 0, so the read address channel does not initiate a read transaction.
- **AWADDR** = 2024 and **ARADDR** = 2024 are present, but they are ignored because their corresponding VALID signals are low.
- **AWREADY** and **ARREADY** may remain asserted, but no handshake occurs because **VALID** = 0.
- **WDATA** remains inactive because no write transaction is started.
- **WVALID** = 0, so no write data transfer occurs.
- **RVALID** = 0, so no read data is returned.
- **BVALID** = 0, so no write response is generated.
- **RRESP** = 0 and **BRESP** = 0 remain at their idle values.
- Memory enable/write signals remain inactive, indicating that no memory access occurs.
- The AXI interface remains in the idle state without performing any read or write operation.

Result: With both **AWVALID** and **ARVALID** deasserted, the AXI slave correctly performs no read or write transaction. Test case passed as expected.

Timing diagram for the first read operation. The diagram shows signals for address, data, and ready status over time. The address (WRT_ADDR) is 16'd103. The data (WRT_DATA) is 32'h9657f62. The ready signal (WRT_READY) is high for the duration of the operation. The read operation is completed at 196,700,000 ps.

This test case performs 10 random AXI write transactions using constrained-random stimulus. A constraint is applied to ensure that the write burst does not cross a 4 KB address boundary. For all 10 transactions, the write address, burst length, and write data are randomized within the allowed constraints.

6.6 Random read operations without 4 KB boundary crossing:



The waveform shows multiple random AXI read transactions with randomized read addresses and burst lengths. The transactions complete successfully while satisfying the 4 KB boundary constraint:

53

6.7 Random write operations with 4 KB boundary crossing:

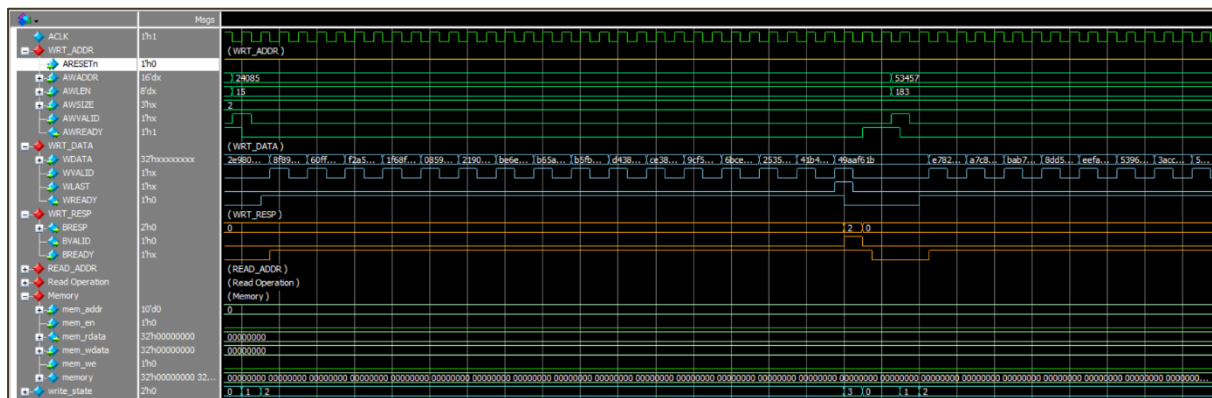


Figure 14: Random write operations with 4KB boundary crossing test.

- The write burst crosses the 4 KB address boundary, as indicated by the boundary-crossing condition.
 - **BRESP** = 2'b10, indicating a SLVERR response due to the invalid boundary-crossing write burst.
 - The write state transitions through the expected states and completes the transaction.
- Result: The randomized write transaction intentionally crosses the 4 KB boundary, and the AXI interface correctly detects the boundary violation and returns **BRESP** = 2'b10 (SLVERR). Test case passed as expected.

```
# ===== END OF SIMULATION =====
# ===== SUMMARY =====
# [RATE]:          Total Pass Rate: 100%
# [PASS COUNT]:    28836
# [FAIL COUNT]:    0
# [TOTAL CHECKS]:  3204
# =====
#
# ===== NO ERRORS LOGGED =====
```

6.8 Random read operations with 4 KB boundary crossing:

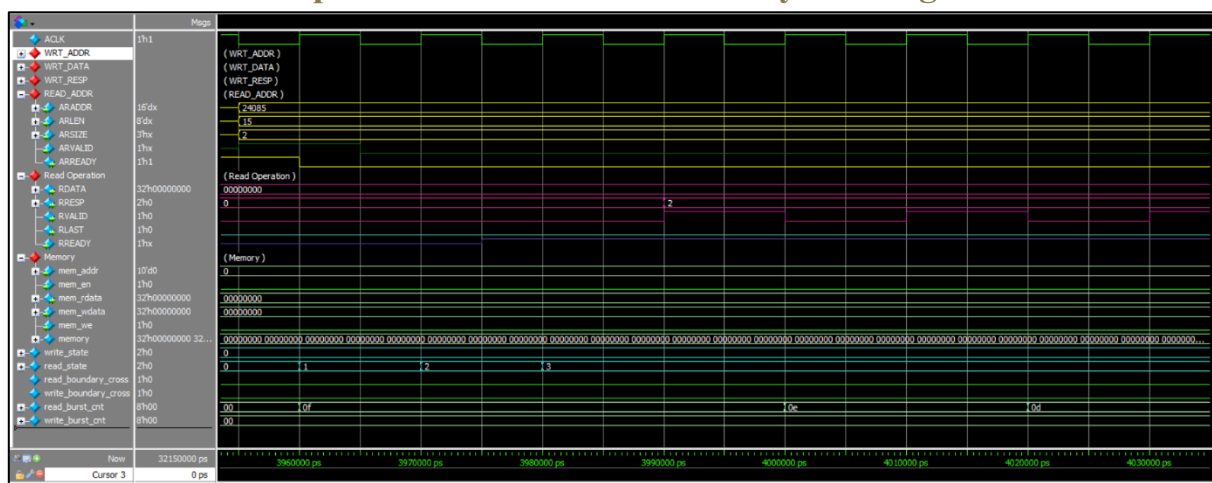


Figure 15: Random read operations with 4 KB boundary crossing test.

Result: The randomized read transaction crosses the 4 KB boundary, and the AXI interface correctly detects the violation and generates $RRESP = 2'b10$ (SLVERR). Test case passed as expected.

```
# ===== END OF SIMULATION =====
# ===== SUMMARY =====
# [RATE]:          Total Pass Rate: 100%
# [PASS COUNT]:    28890
# [FAIL COUNT]:    0
# [TOTAL CHECKS]:  3210
#
# ===== NO ERRORS LOGGED =====
#
```

6.9 Random operations with and without 4 KB boundary crossing:

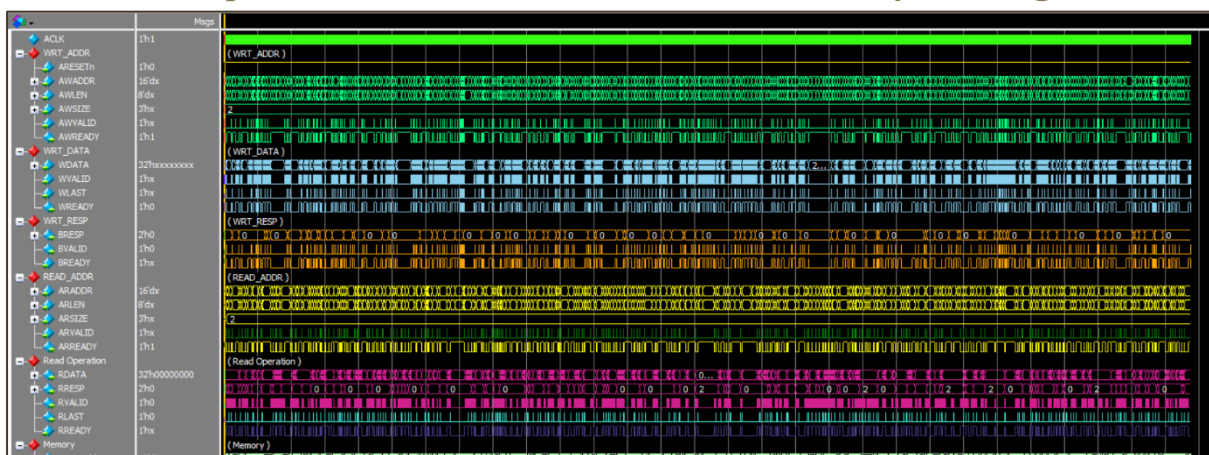


Figure 16: Random operations with and without 4 KB boundary crossing test

A total of 800 AXI transactions were generated using constrained-random verification. All the constraints defined in the AXI transaction section were enabled and applied during randomization, covering different addresses, burst lengths, transaction types, and boundary conditions.

The large number of randomized transactions provides broad functional coverage of the AXI interface and verifies its behavior under different valid and invalid scenarios. The scoreboard continuously checks the DUT behavior on each clock cycle, resulting in a large number of total checks compared with the number of generated transactions.

Result: The DUT was extensively verified using 800 constrained-random AXI transactions, with the defined constraints applied throughout the test. Test passed as expected.

```
# ===== END OF SIMULATION =====
# ===== SUMMARY =====
# [RATE]:          Total Pass Rate: 100%
# [PASS COUNT]:    1293741
# [FAIL COUNT]:    0
# [TOTAL CHECKS]:  143749
#
# ===== NO ERRORS LOGGED =====
#
```

6.10 Randomization of ARESETn during the simulation:

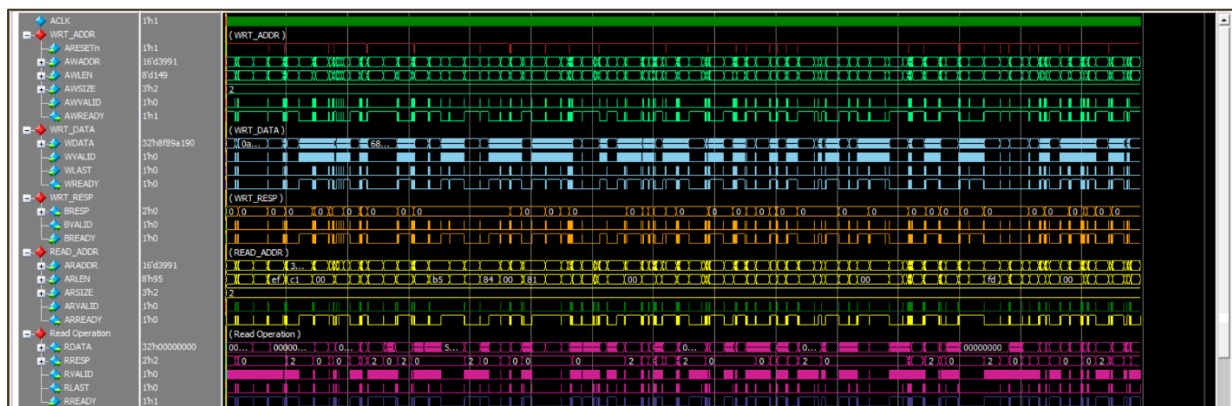


Figure 17: Randomization of ARESETn during the simulation test.

We enabled the constraint of ARESETn randomization with distribution of 95% OFF and 5% ON, with the following results:

```
# =====
# ===== END OF SIMULATION =====
# ===== SUMMARY =====
# [RATE]:          Total Pass Rate: 100%
# [PASS COUNT]:    269253
# [FAIL COUNT]:    0
# [TOTAL CHECKS]:  29917
# =====
```

7. Coverage:

7.1 Code coverage exclude:

7.1.1 FSM exclusion:

```
coverage exclude -scope /AXI_tb/DUT -ftrans write_state W_ADDR->W_IDLE
coverage exclude -scope /AXI_tb/DUT -ftrans write_state W_DATA->W_IDLE
coverage exclude -scope /AXI_tb/DUT -ftrans read_state R_ADDR->R_IDLE
coverage exclude -scope /AXI_tb/DUT -ftrans read_state R_WAIT->R_IDLE
```

the write FSM follows a strict linear flow:

W_IDLE -> W_ADDR -> W_DATA -> W_RESP -> W_IDLE.

It never jumps directly from W_ADDR or W_DATA back to W_IDLE.

Similarly, the read FSM follows R_IDLE -> R_ADDR -> R_WAIT -> R_DATA -> R_IDLE.

It never jumps directly from R_ADDR or R_WAIT back to R_IDLE. Because these direct transitions do not exist in the RTL logic, coverage engines flag them as missing transitions unless explicitly excluded.

7.1.2 Toggle exclusion:

```
coverage exclude -scope /AXI_tb/DUT -togglenode ARSIZE
coverage exclude -scope /AXI_tb/DUT -togglenode AWSIZE
coverage exclude -scope /AXI_tb/DUT -togglenode BRESP
coverage exclude -scope /AXI_tb/DUT -togglenode read_addr_incr
coverage exclude -scope /AXI_tb/DUT -togglenode read_size
coverage exclude -scope /AXI_tb/DUT -togglenode RRESP
coverage exclude -scope /AXI_tb/DUT -togglenode write_addr_incr
coverage exclude -scope /AXI_tb/DUT -togglenode write_size
coverage exclude -scope /AXI_tb/DUT/mem_inst -togglenode j
```

- AWSIZE / ARSIZE / write_size / read_size: In my driver, AWSIZE and ARSIZE are hardcoded to 'd2 (which means 4 bytes / 32 bits per transfer). Because AWSIZE is fixed to 3'b010, bits 0 and 2 never toggle to 1, leaving them dead in toggle coverage.
- write_addr_incr / read_addr_incr: Since size is fixed to 4 bytes (3'b010), (1 << 2) equals 3'b100. Bits 0 and 1 remain tied to 0 and never toggle.
- BRESP / RRESP: Represent AXI response codes (2'b00 = OKAY, 2'b10 = SLVERR). In nominal non-error testing, these signals remain 2'b00, meaning bit 1 never toggles unless a slave error occurs.
- mem_inst/j: A loop iterator or dummy variable inside the memory instance that doesn't represent real hardware flip-flops or toggling bus wires.

7.1.3 Line exclusion:

```
coverage exclude -src axi4.v -scope /AXI_tb/DUT -line 288 -code s
coverage exclude -src axi4.v -line 212 -code s
coverage exclude -src axi4.v -scope /AXI_tb/DUT -line 204 -code b
coverage exclude -src axi4.v -scope /AXI_tb/DUT -line 212 -code b
coverage exclude -src axi4.v -scope /AXI_tb/DUT -line 288 -code b
coverage exclude -src axi4.v -scope /AXI_tb/DUT -line 153 -code c
coverage exclude -src axi4.v -scope /AXI_tb/DUT -line 173 -code c
coverage exclude -src axi4.v -scope /AXI_tb/DUT -line 184 -code c
coverage exclude -src axi4.v -scope /AXI_tb/DUT -line 189 -code c
coverage exclude -src axi4.v -scope /AXI_tb/DUT -line 204 -code c
coverage exclude -src axi4.v -scope /AXI_tb/DUT -line 224 -code c
coverage exclude -src axi4.v -scope /AXI_tb/DUT -line 264 -code c
```

Line 288 (default: read_state <= R_IDLE;):

- **Justification:** The Read FSM uses a 2-bit state register (reg [1:0] read_state) to represent exactly 4 valid states (R_IDLE, R_ADDR, R_WAIT, R_DATA). Since all $2^2 = 4$ state combinations are fully defined in the case statement, the default branch is unreachable hardware logic.

Line 212 (default: write_state <= W_IDLE;):

- **Justification:** The Write FSM similarly uses a 2-bit state register covering all 4 defined states (W_IDLE, W_ADDR, W_DATA, W_RESP). The default branch is impossible to hit during simulation.

Line 204 (if (BREADY && BVALID)):

- **Justification:** The driver protocol pre-asserts BREADY early in anticipation of the response. When the FSM enters W_RESP and evaluates this condition, BREADY is always high. Consequently, the branch where BREADY == 0 inside this evaluation is never taken.

Lines 212 & 288 (default branches):

- **Justification:** Waiving the unreachable default branches for both Write and Read state machines as detailed in the statement coverage analysis.

Line 153 (if (AWVALID && AWREADY)) & Line 224 (if (ARVALID && ARREADY)):

- **Justification:** The DUT pre-asserts AWREADY / ARREADY high in IDLE states (AWREADY = 1, ARREADY = 1). When evaluating these conditions, READY is always high, meaning the condition sub-expression READY == 0 is never exercised while entering the state.

Line 173 (if (WVALID && WREADY)):

- **Justification:** WREADY is driven high by the FSM upon entering W_DATA before sampling data. Thus, WREADY is already high when WVALID arrives, preventing the WREADY == 0 sub-condition from firing.

Line 184 (if (WLAST || write_burst_cnt == 0)):

- **Justification:** SystemVerilog performs short-circuit evaluation. The driver strictly asserts WLAST = 1 on the final beat of every burst. Because WLAST evaluates to 1 first, the simulator short-circuits the expression and skips evaluating write_burst_cnt == 0. This leaves the second condition false-evaluation unhit, despite write_burst_cnt reaching zero naturally.

Line 189 (if (!write_addr_valid || write_boundary_cross)):

- **Justification:** Short-circuit evaluation waiver where the first condition evaluation satisfies the check, preventing sub-condition coverage completion.

Line 204 (if (BREADY && BVALID)):

- **Justification:** BREADY is pre-asserted by the driver, so the condition BREADY == 0 is never observed during response evaluation.

Line 264 (if (RREADY && RVALID)):

- **Justification:** The driver pre-asserts RREADY high during read data loops. Therefore, the condition is evaluated with RREADY permanently high, leaving the RREADY == 0 evaluation path un-covered.

7.2 Code coverage:

```

432
433
434   Total Coverage By Instance (filtered view): 100.00%
435
436

```

7.3 Functional and cross coverage:

1	COVERGROUP COVERAGE:			
2	-----			
3	Covergroup	Metric	Goal	Status
4	-----			
5				
6	TYPE /axi_transaction_pkg/axi_transaction/cg	100.0%	100	Covered
7	covered/total bins:	57	57	
8	missing/total bins:	0	57	
9	% Hit:	100.0%	100	
10	Coverpoint cg::cp_awaddr	100.0%	100	Covered
11	covered/total bins:	3	3	
12	missing/total bins:	0	3	
13	% Hit:	100.0%	100	
14	bin low_addr	85	1	Covered
15	bin mid_addr	227	1	Covered
16	bin high_addr	333	1	Covered
17	Coverpoint cg::cp_awlen	100.0%	100	Covered
18	covered/total bins:	4	4	
19	missing/total bins:	0	4	
20	% Hit:	100.0%	100	
21	bin len_single	650	1	Covered
22	bin len_short	8	1	Covered
23	bin len_med	61	1	Covered
24	bin len_max	81	1	Covered
25	Coverpoint cg::cp_araddr	100.0%	100	Covered
26	covered/total bins:	3	3	
27	missing/total bins:	0	3	
28	% Hit:	100.0%	100	
29	bin low_addr	85	1	Covered
30	bin mid_addr	227	1	Covered
31	bin high_addr	333	1	Covered
32	Coverpoint cg::cp_arlen	100.0%	100	Covered
33	covered/total bins:	4	4	
34	missing/total bins:	0	4	
35	% Hit:	100.0%	100	
36	bin len_single	650	1	Covered
37	bin len_short	8	1	Covered
38	bin len_med	61	1	Covered
39	bin len_max	81	1	Covered
40	Coverpoint cg::cp_arvalid	100.0%	100	Covered
41	covered/total bins:	2	2	
42	missing/total bins:	0	2	
43	% Hit:	100.0%	100	
44	bin auto[0]	515	1	Covered
45	bin auto[1]	285	1	Covered

46	Coverpoint cg::cp_arvalid	100.0%	100	Covered
47	covered/total bins:	2	2	
48	missing/total bins:	0	2	
49	% Hit:	100.0%	100	
50	bin auto[0]	243	1	Covered
51	bin auto[1]	557	1	Covered
52	Coverpoint cg::cp_op	100.0%	100	Covered
53	covered/total bins:	3	3	
54	missing/total bins:	0	3	
55	% Hit:	100.0%	100	
56	bin auto[WRITE]	285	1	Covered
57	bin auto[READ]	272	1	Covered
58	bin auto[IDLE]	243	1	Covered
59	Coverpoint cg::cp_address_mode	100.0%	100	Covered
60	covered/total bins:	3	3	
61	missing/total bins:	0	3	
62	% Hit:	100.0%	100	
63	bin mode_0	244	1	Covered
64	bin mode_1	323	1	Covered
65	bin mode_2	155	1	Covered
66	Cross cg::cp_op_mode	100.0%	100	Covered
67	covered/total bins:	9	9	
68	missing/total bins:	0	9	
69	% Hit:	100.0%	100	
70	bin <auto[WRITE],mode_0>	86	1	Covered
71	bin <auto[READ],mode_0>	89	1	Covered
72	bin <auto[IDLE],mode_0>	69	1	Covered
73	bin <auto[WRITE],mode_1>	123	1	Covered
74	bin <auto[READ],mode_1>	107	1	Covered
75	bin <auto[IDLE],mode_1>	93	1	Covered
76	bin <auto[WRITE],mode_2>	52	1	Covered
77	bin <auto[READ],mode_2>	53	1	Covered
78	bin <auto[IDLE],mode_2>	50	1	Covered
79	Cross cg::cp_wadd_len	100.0%	100	Covered
80	covered/total bins:	12	12	
81	missing/total bins:	0	12	
82	% Hit:	100.0%	100	
83	bin <mid_addr,len_single>	174	1	Covered
84	bin <high_addr,len_single>	281	1	Covered
85	bin <low_addr,len_short>	4	1	Covered
86	bin <mid_addr,len_short>	1	1	Covered
87	bin <high_addr,len_short>	3	1	Covered
88	bin <low_addr,len_med>	11	1	Covered
89	bin <low_addr,len_max>	7	1	Covered
90	bin <mid_addr,len_med>	24	1	Covered
91	bin <mid_addr,len_max>	28	1	Covered
92	bin <high_addr,len_med>	15	1	Covered
93	bin <high_addr,len_max>	34	1	Covered
94	bin low_singlelen	63	1	Covered

95	Cross cg::cp_radd_len	100.0%	100	Covered
96	covered/total bins:	12	12	
97	missing/total bins:	0	12	
98	% Hit:	100.0%	100	
99	bin <mid_addr,len_single>	174	1	Covered
00	bin <high_addr,len_single>	281	1	Covered
01	bin <low_addr,len_short>	4	1	Covered
02	bin <mid_addr,len_short>	1	1	Covered
03	bin <high_addr,len_short>	3	1	Covered
04	bin <low_addr,len_med>	11	1	Covered
05	bin <low_addr,len_max>	7	1	Covered
06	bin <mid_addr,len_med>	24	1	Covered
07	bin <mid_addr,len_max>	28	1	Covered
08	bin <high_addr,len_med>	15	1	Covered
09	bin <high_addr,len_max>	34	1	Covered
10	bin low_singlelen	63	1	Covered
11	CLASS axi_transaction			
12				
13	TOTAL COVERGROUP COVERAGE: 100.0% COVERGROUP TYPES: 1			

AWREADY = 0.

ARREADY = 0.